

CYB 125 Final Project Specification

Windows Baseline Configuration Snapshot Collector

Objective

You will be building a python script that captures a snapshot of your Windows VM's configuration and saves it as a JSON file. A security analyst could later compare your snapshot against a future snapshot of the same machine to detect unauthorized changes. This practice is called system baselining, and it is one of the highest-value defensive controls a small security team can implement.

Throughout the process of building this script, you will be instructed on how to use AI in a multi-step process, and in a way that you're always aware of what part of the process you're currently in and you understand what the AI is working on and producing, rather than attempting to one-shot the whole project and have the AI produce code that you don't understand and can't explain.

Important: This project specification is a walk-through of the project, meaning I will be showing you how to complete each aspect of this project. Please, follow this project specification from top to bottom, line by line, and do not skip any information. All of the information you need to complete the project is in this specification! This project should take you no more than 4-6 hours. If you get stuck for longer than an hour please contact me!

How the Project and Project Tasks are Organized

You will be guided to perform this project in 4 distinct parts:

Part 0 is the setup phase. You will be guided to set up a GitHub repository for this project, setup the VLE so that you can login to GitHub and connect to your repository. You will then verify that Copilot AI features are enabled and that you can get a chat response from Copilot.

Part 1 is the planning phase. This project includes a design phase where you will document your understanding of the project specifications, list and describe the configurations that you will be collections. **AI is not allowed for this part and all answers must be provided by you.**

Part 2 is the development phase. You will be guided in how to use AI to develop your script. Part 2 has eight milestones, where each milestone:

- Reviews one or two Python concepts you'll need
- Adds one new section (or two related sections) to your baseline configuration snapshot
- Ends with a working python main.py that produces a slightly bigger JSON file than it did before
- Has a short self-check and a one-paragraph reflection question.

Part 3 is the Final Reflection section.

Part 0 — Setup the Project

Part 0. Step 0 — Log in to the VLE

You will need to build and run your script inside of the VLE to have access to the Windows objects necessary for your script to run.

Step 0.1: Log in to the VLE at <https://portal.cyber.cochise.edu> using the first part of your email (before the @) and the same password you use to log into your email and click “CYB 125/210 Programming”

Step 0.2: Download CYB125_FinalProject.zip to your VLE from moodle.

Step 0.3: Unzip the file by right-clicking it in File Explorer and selecting “Extract All”

Step 0.4: Choose a destination to extract the files to (C:\Users\Student\Downloads) and click “Extract”

Provided files:

File	Instructions
CYB125_FinalProject.zip	This is a zip file that contains all of the files below. Download it from Moodle and unzip it by right-clicking the file in File Explorer and selecting "Extract All".
helpers.py	This is the main file you will need to modify. Add your code under each of the 8 Milestone comments.
main.py	You will also need to modify this file once each Milestone, just to uncomment the code that calls your helper functions.
example.output.baseline.json	This file is for your reference only and does not need to be modified or ingested by your script in any way. This file contains sample output, similar to what your program will need to produce, which you can use to plan your program and visualize how your program output will need to be structured.
README.md	A starter README.md file that you will need to complete and submit for your first requirement.
CYB 125 - Final Project Spec.pdf	This file, I know its long, but it's a set of instructions and if you follow them as they're laid out here then you will save yourself a lot of time . This project should take you no more than 4-6 hours. If you get stuck for longer than an hour please contact me!

Part 0. Step 1 — Setup the VLE (Install Git, GitHub, Copilot AI)

Because we'll be using VS Code AI Copilot features, you will need to sign-up and sign-in to github to complete this project. It doesn't matter if you use your student or personal email address to sign up as you can link both if you want to access this repo again after you graduate.

Step 1.1: Inside of the VLE, open a browser and navigate to www.github.com.

Step 1.2: If you are not registered for github, please do so, then login.

Step 1.3: From the dashboard, click "New" to create a new repository.

- Name the new repository **CYB125-ai-final-project**.
- Make sure "Add README" is set to On,
- Make sure the repo is set to public,
- and then click "Create repository."

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * prof0x / Repository name * CYB125-ai-final-project
✓ CYB125-ai-final-project is available.

Great repository names are short and memorable. How about [refactored-octo-invention](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Start with a template
Templates pre-configure your repository with files. No template

Add README
READMEs can be used as longer descriptions. [About READMEs](#) On

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

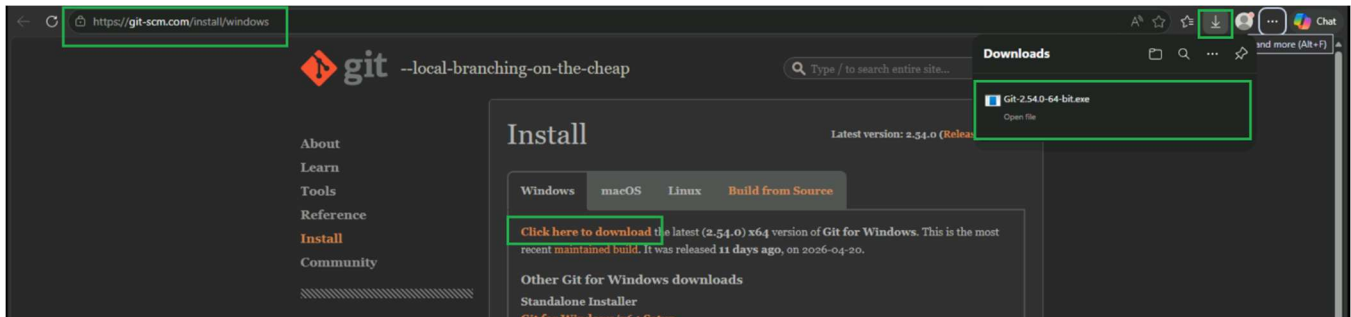
Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

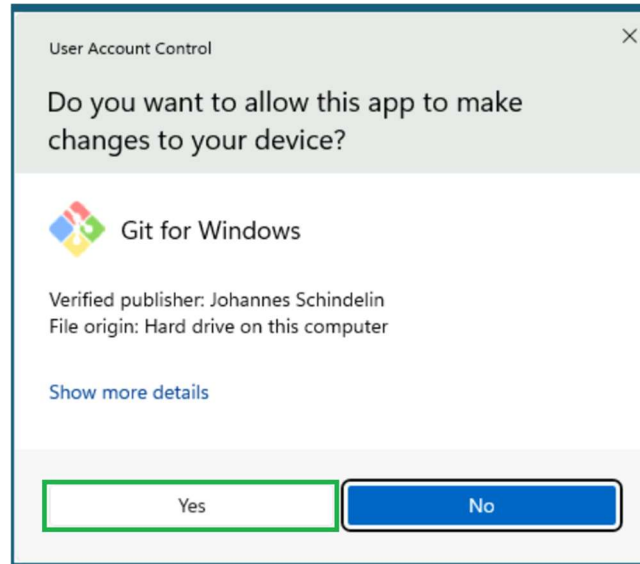
Step 1.4: Your VLE does not come pre-configured with the github or git clients so you will have to download and install it. I do apologize, this part is long so I will go over this in class and post a video of the setup process. Please reach out to me asap if you have any issues with this part.

Step 1.5: Inside of your VLE, navigate to the following url <https://git-scm.com/install/windows>.

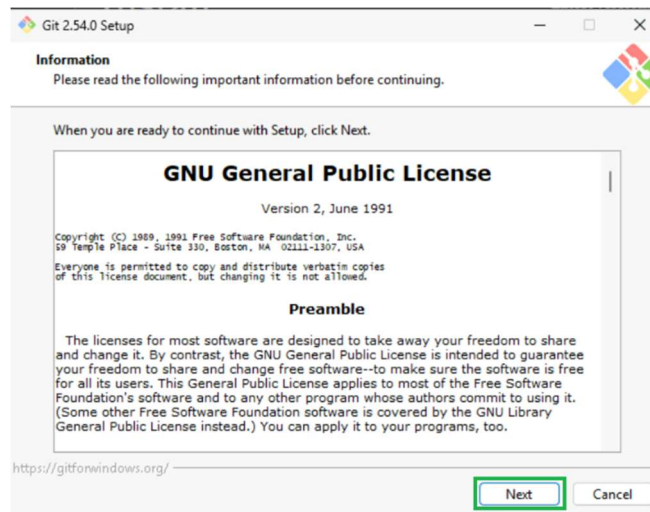
- Click where it says “**Click here to download** the latest (2.54.0) x64 version of Git for Windows.”
- Open your Downloads folder and double-click the “Git-2.54.0-64bit” installer to launch the installation process. **There are 17 total steps to this part. If it makes you feel any better I had to go through each one too, take screenshots, and put this document together. Hang in there!**



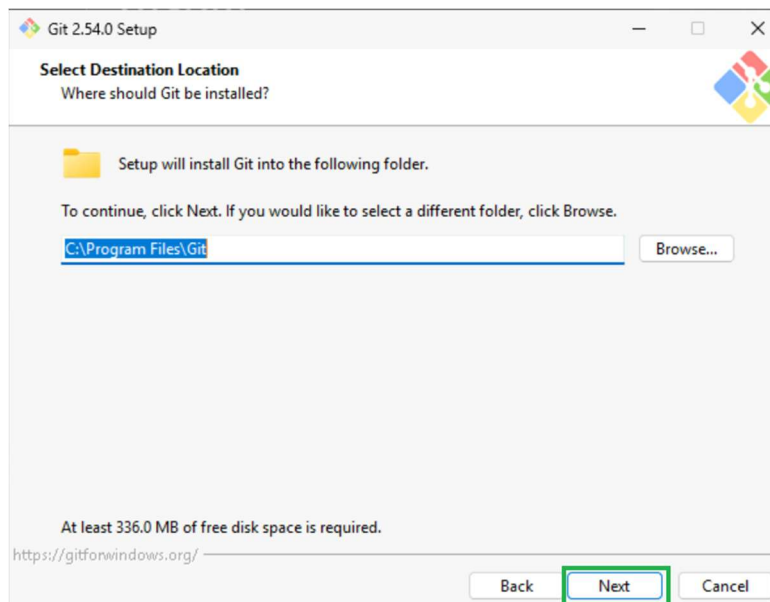
- Click “Yes” when asked if you want to allow this app to make changes to your device.



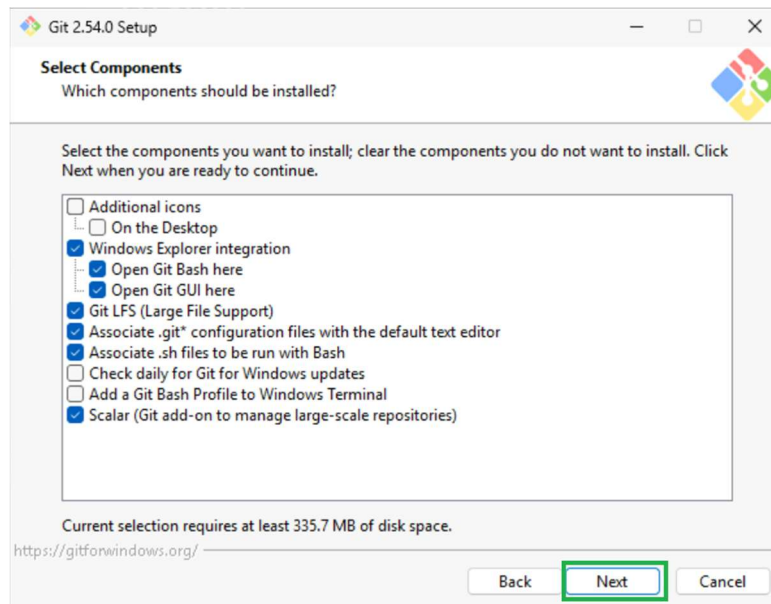
- Click “Next” to accept the GNU General Public License



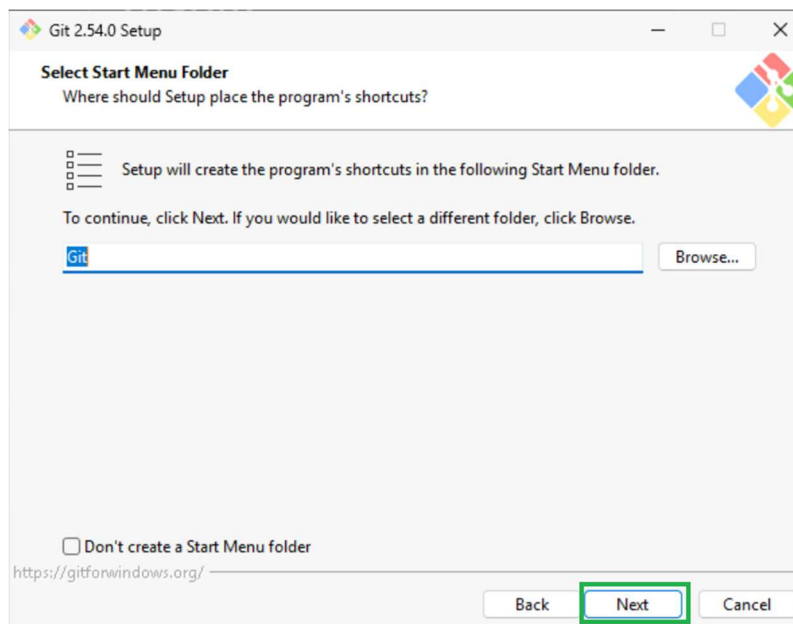
- Click “Next” to select the default installation location for Git.



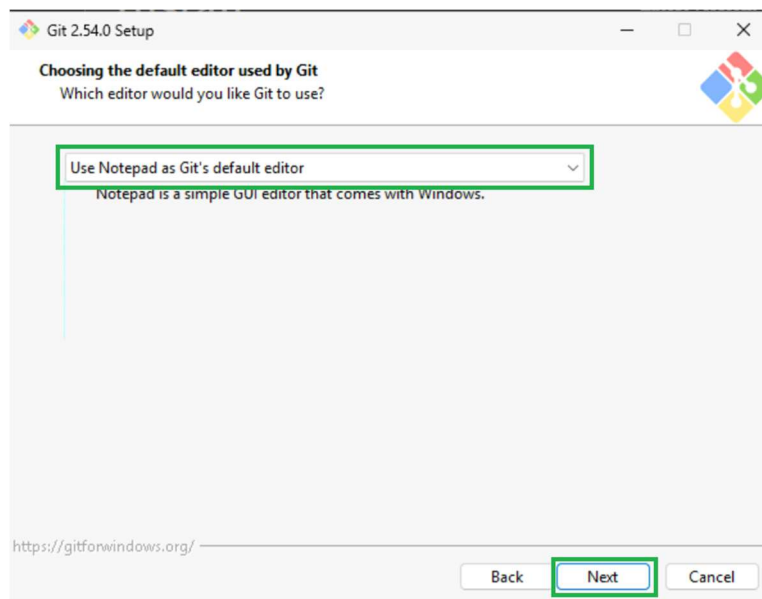
- Click “Next” to select the default components to be installed



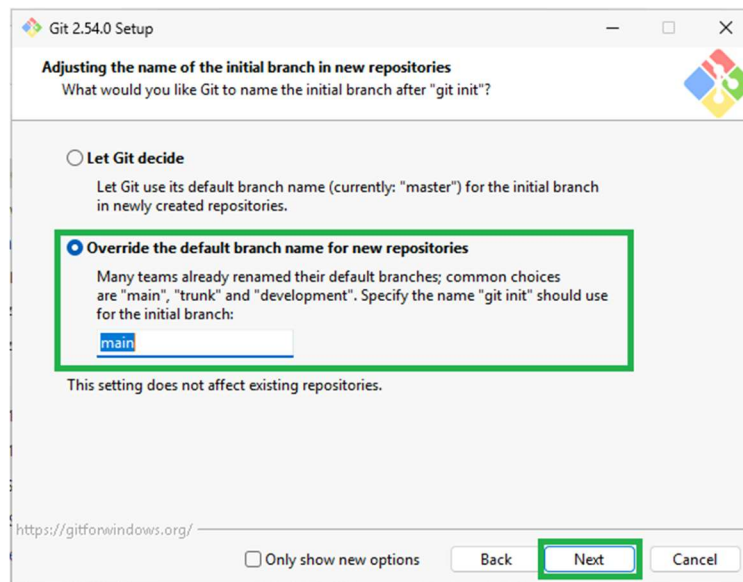
- Click “Next” to select the default Start Menu Folder for Git



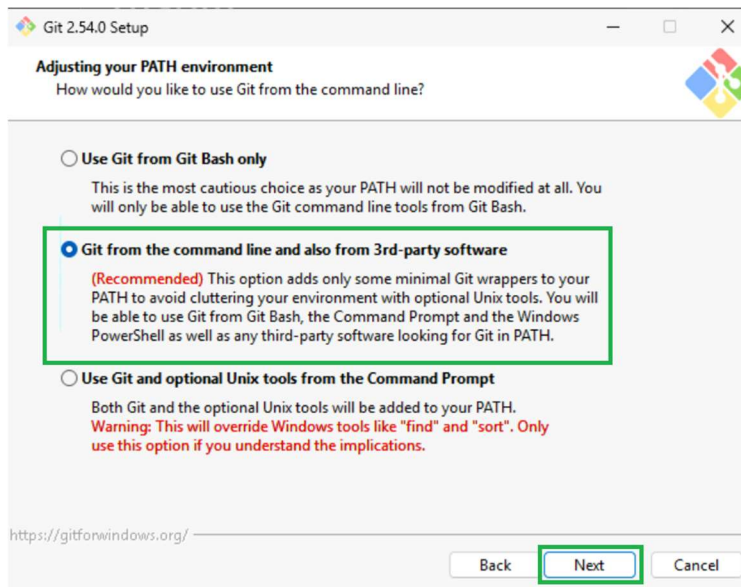
- Make sure “Use Notepad as Git’s default editor” is selected otherwise things get annoying to manage down the line. Click “Next”



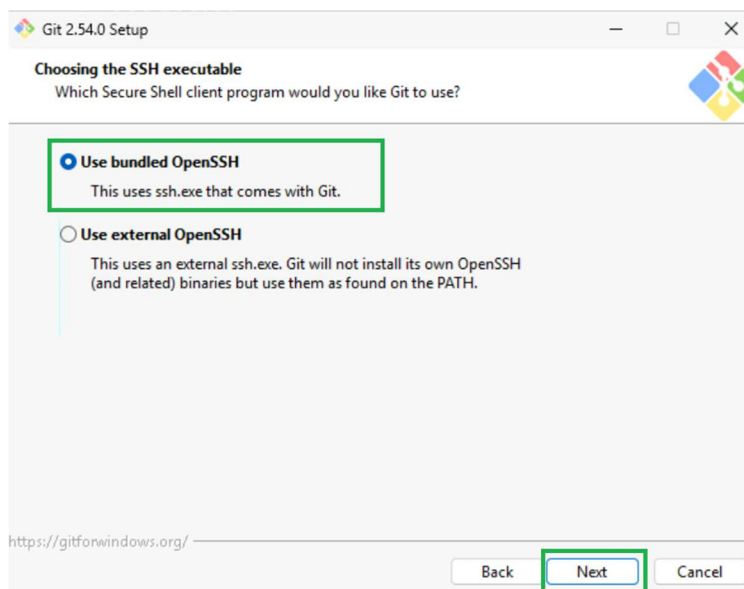
- Select “Override the default branch name for new repositories” and make sure the initial branch is called “main”



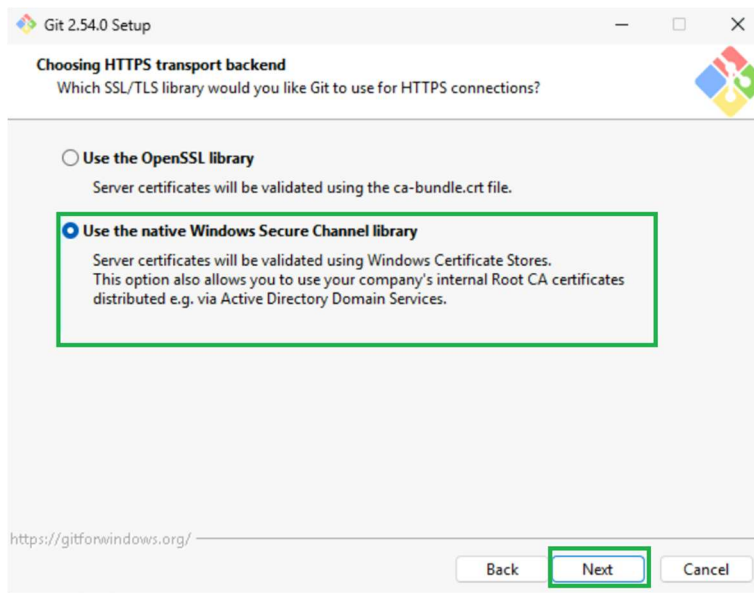
- Make sure “Git from the command line and also from 3rd party software” is selected and click “Next”.



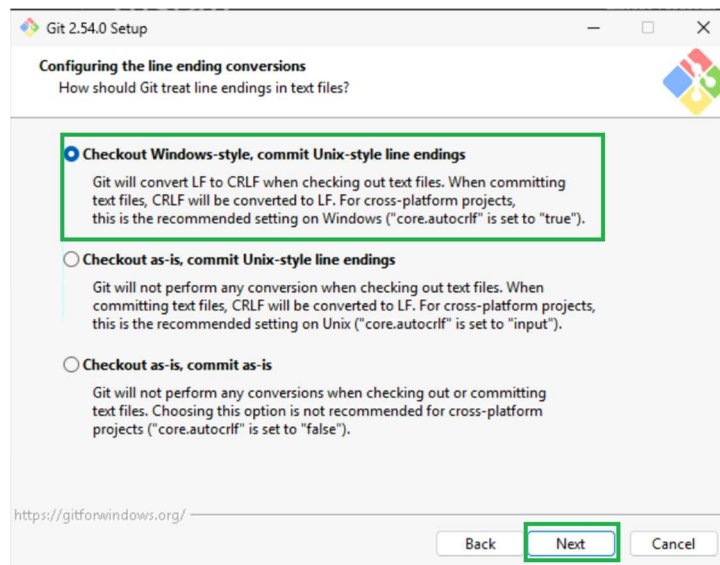
- Make sure “Use bundled OpenSSH” is selected and click “Next”



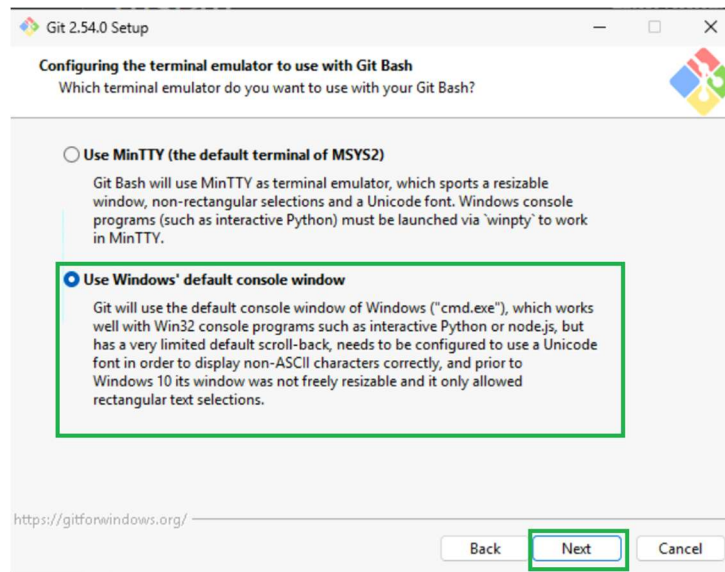
- Make sure “Use the native Windows Secure Channel library” is selected and click “Next”.



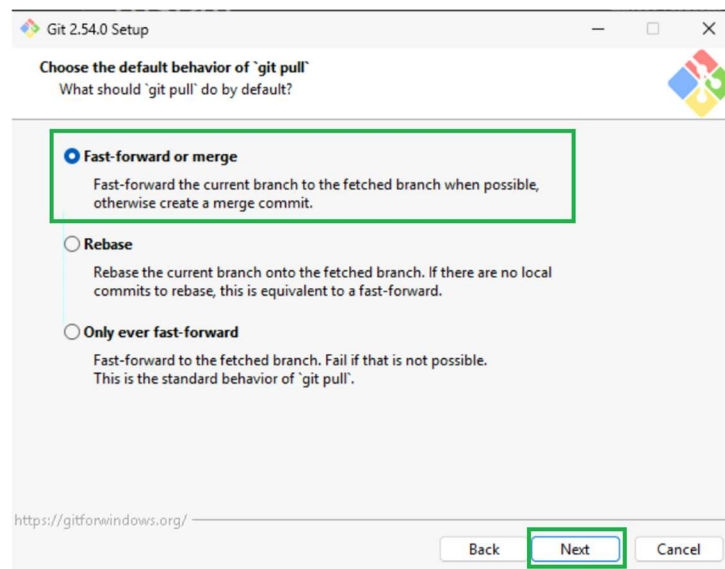
- Make sure “Checkout Windows-style, commit Unix-style line endings” is selected and click “Next”.



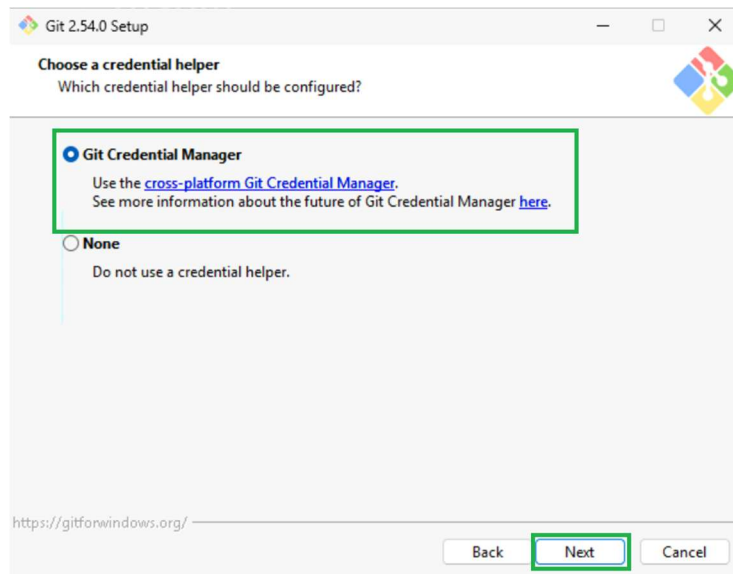
- Select “Use Windows’ default console window” and click “Next”.



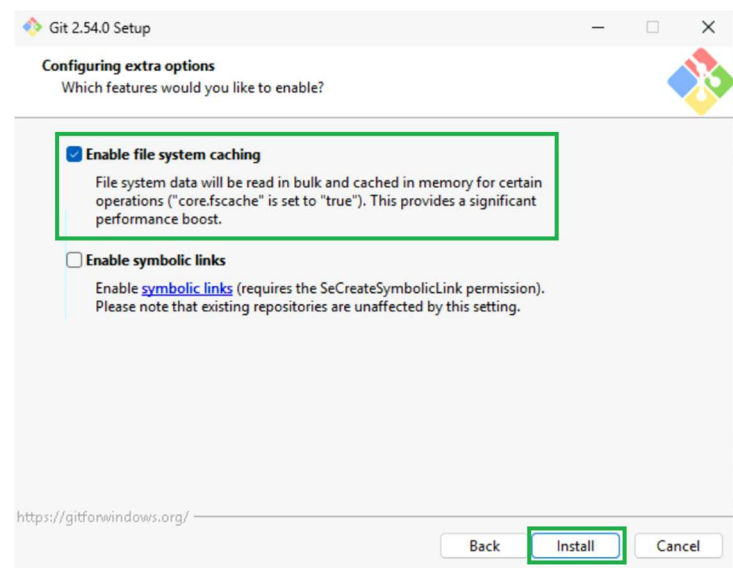
- Make sure “Fast-forward or merge” is selected and click “Next”



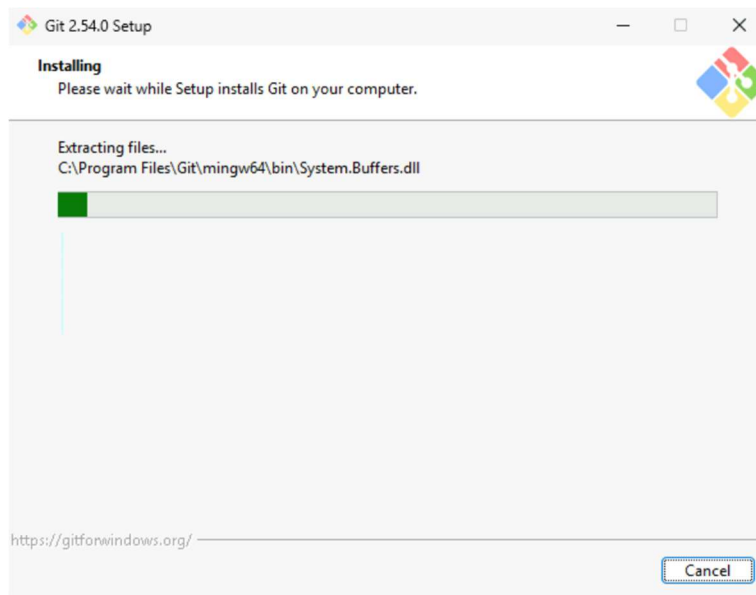
- Make sure “Git Credential Manager” is selected and click “Next”.



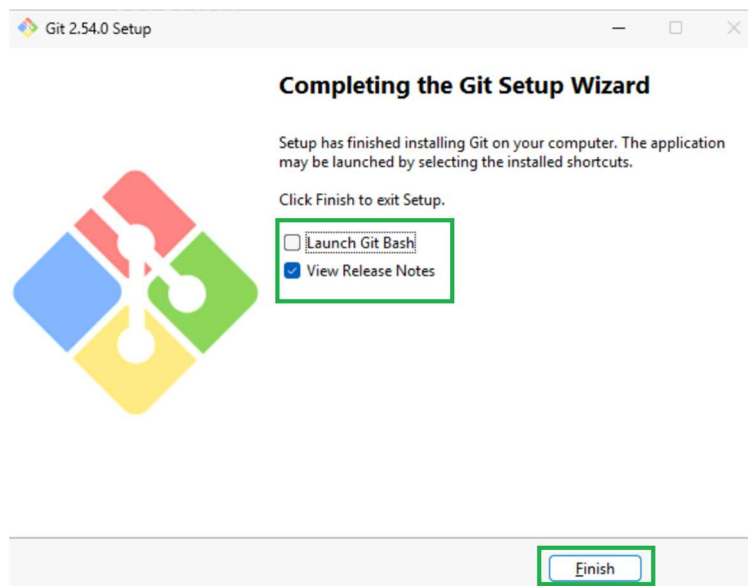
- Make sure “Enable file system caching” is selected and click “Install” to begin the installation process.



- You will see this screen:



- When it's done you will see the following screen. Make sure “Launch Git Bash” is unchecked and click “Finish”.



Ok that parts done! You just installed the git protocol on your VM meaning you now have access to the git command in your command line and in VS Code.

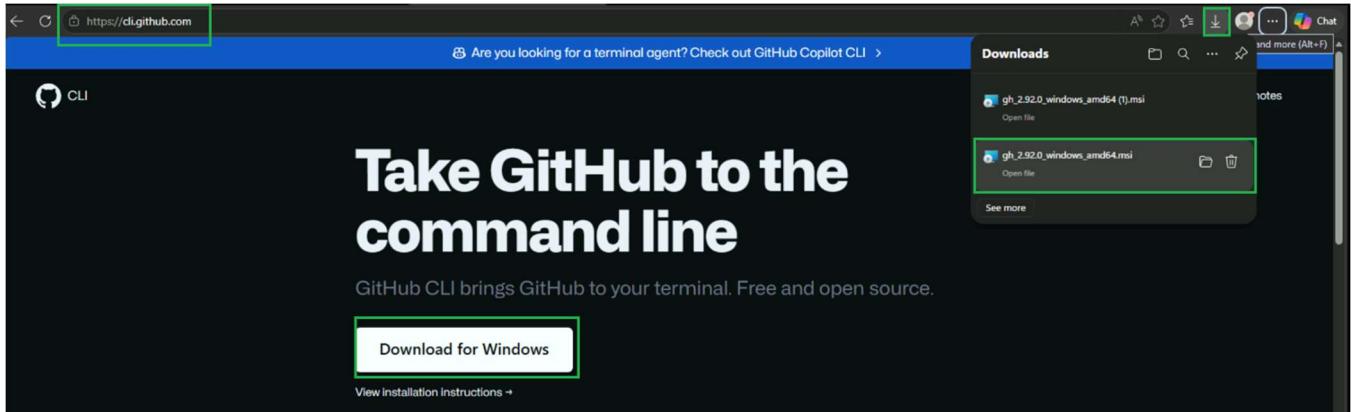
Next we have to install the Github CLI (client).

It shouldn't be a surprise to you that Git and Github are two separate things. Git is a communication protocol, and Github is a social network for developers that uses Git to manage and version software.

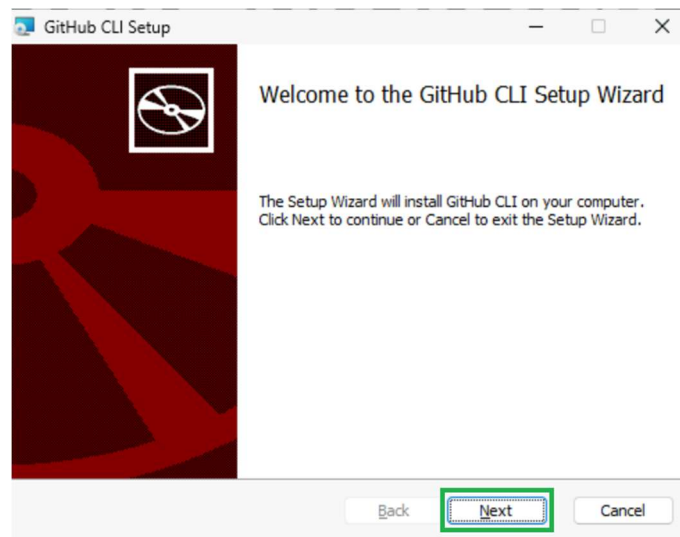
Think of the difference as being similar to how HTTPS is a protocol and a web browser is a client application that uses the HTTPS protocol.

Step 1.6: Inside of your VLE, navigate to the following url <https://cli.github.com/>.

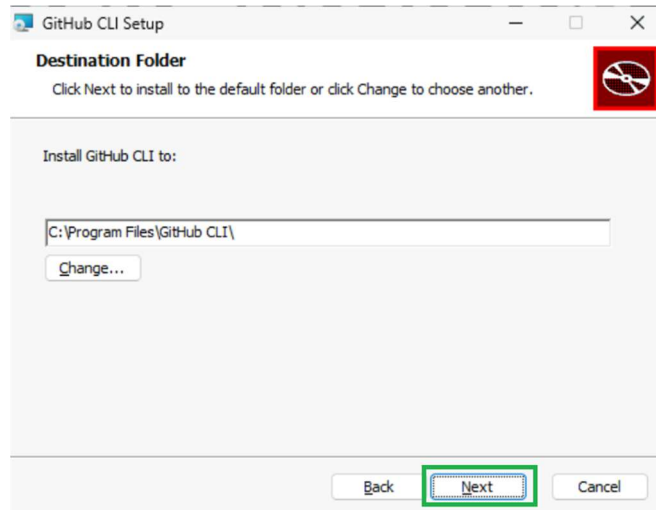
- Click where it says “**Download for Windows.**”
- Open your Downloads folder and double-click the “gh_2.92.0_windows_amd64” installer to launch the installation process.



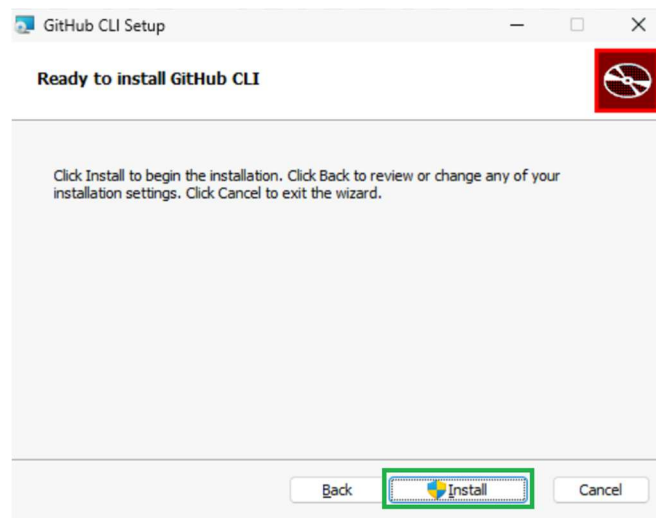
- Click “Next” to begin the setup process.



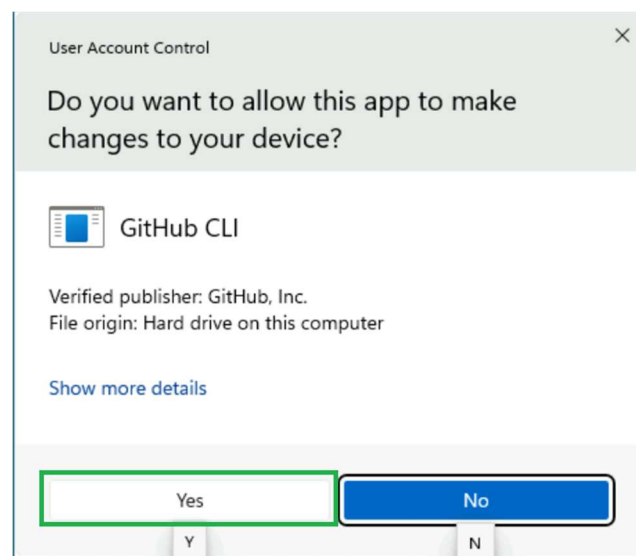
- Click “Next” to select the default installation location for Github CLI.



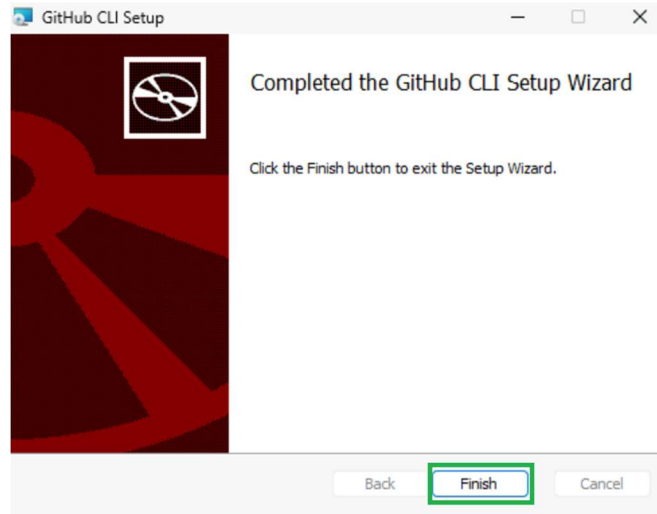
- Click “Install” to begin the installation.



- Click “Yes” to give the installer permissions to make changes to your device.

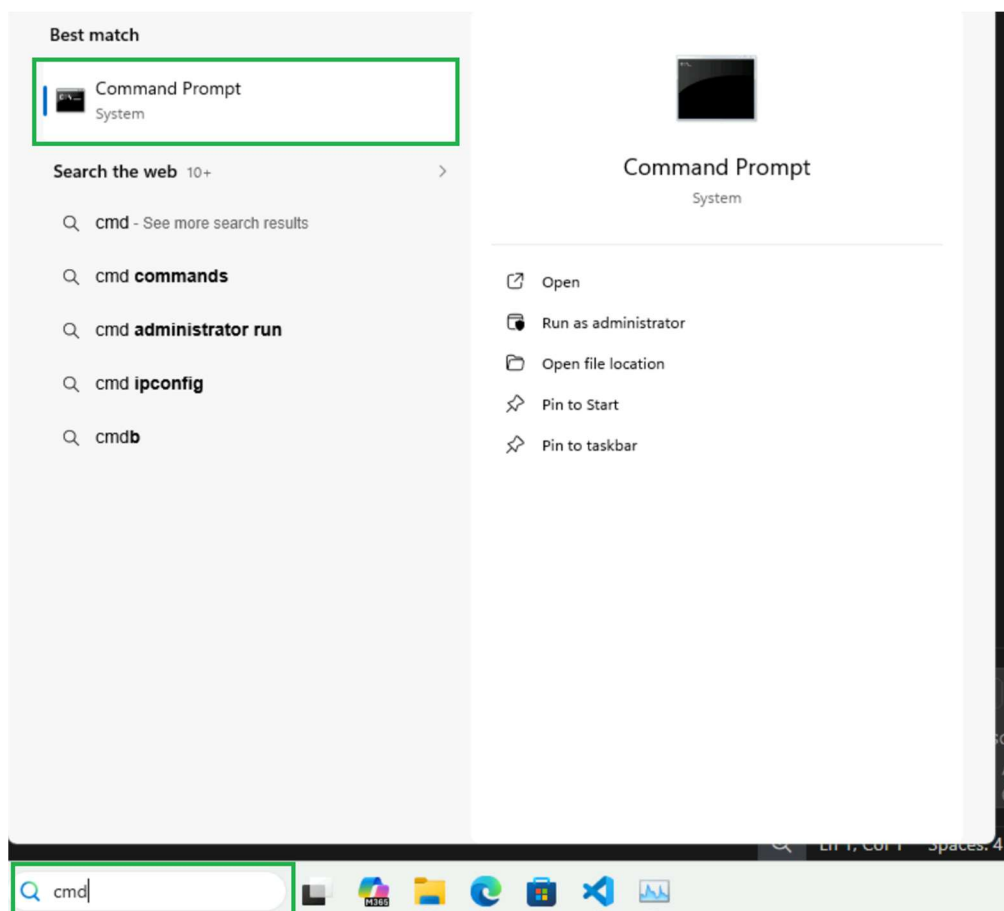


- You will see this screen when the installation is complete. Click “Finish”



Github CLI is not installed on your VM. Let's go use it to log into Github from the command line.

Step 1.7: Inside of your VLE, in the search bar type “cmd” and open a Command Prompt



- In the Command Prompt Window:
 - Verify your current working directory is “C:\Users\Student”

- type the command “mkdir Source”
- type the command “cd Source”
- Verify your current working directory is “C:\Users\Student\Source”

```

Command Prompt
Microsoft Windows [Version 10.0.22631.6936]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Student>mkdir Source
C:\Users\Student>cd Source
C:\Users\Student\Source>

```

- Next type the following commands to configure your github username and email:
 - git config --global user.email cetinerv@cochise.edu
 - git config --global user.name "prof0x"

```

Command Prompt
C:\Users\Student\Source> git config --global user.email "cetinerv@cochise.edu"
C:\Users\Student\Source> git config --global user.name "prof0x"
C:\Users\Student\Source>

```

- Next, type the command “gh auth login” and make the following selections as they appear:
 - GitHub.com
 - HTTPS
 - Yes
 - Login with a web browser
- Copy the one-time code presented to you, then
- click Enter and a browser tab will appear and take you to the device activation screen.

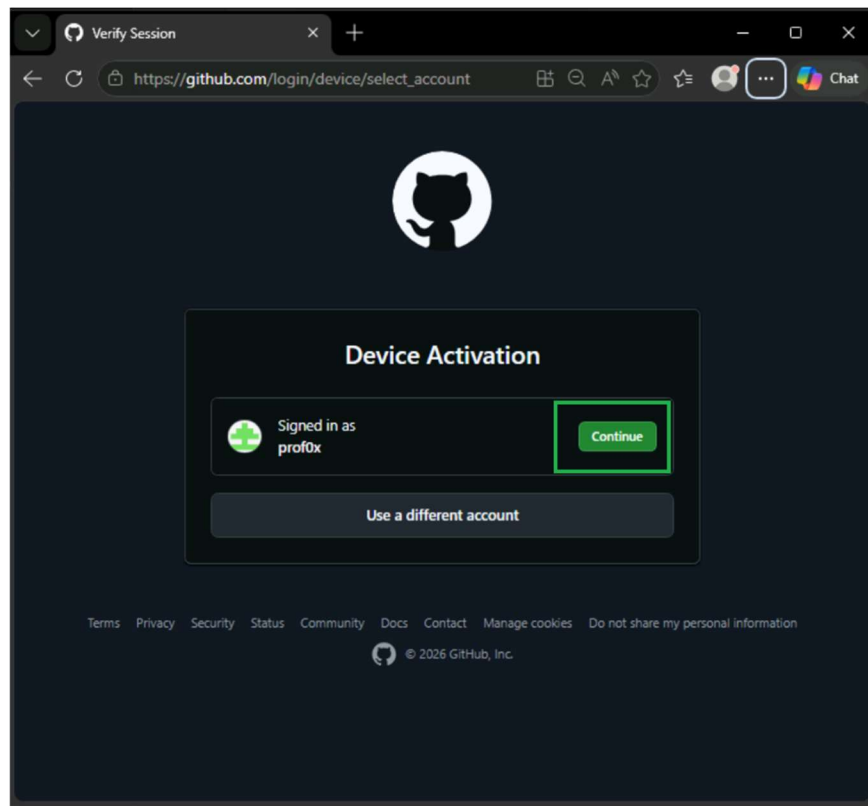
```

Command Prompt - gh auth
C:\Users\Student>mkdir Source
C:\Users\Student>cd Source
C:\Users\Student\Source>gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

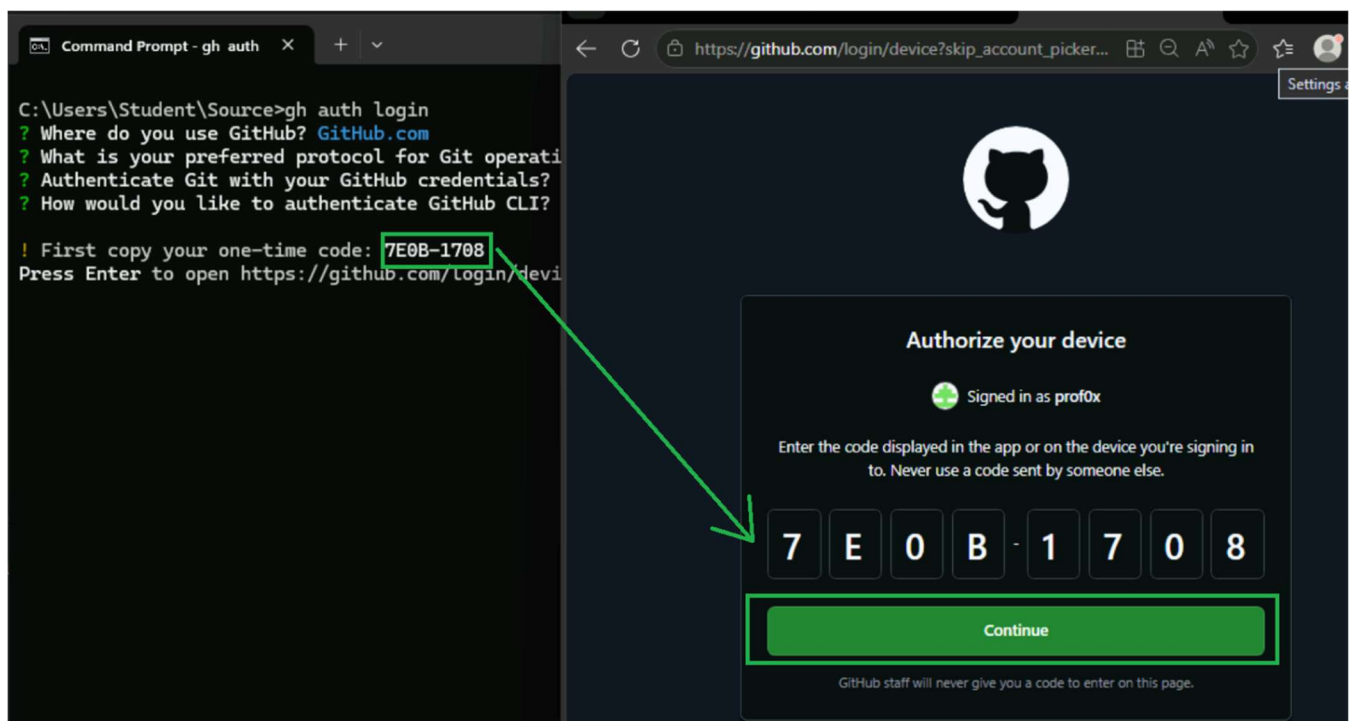
! First copy your one-time code: B012-EB64
Press Enter to open https://github.com/login/device in your browser...

```

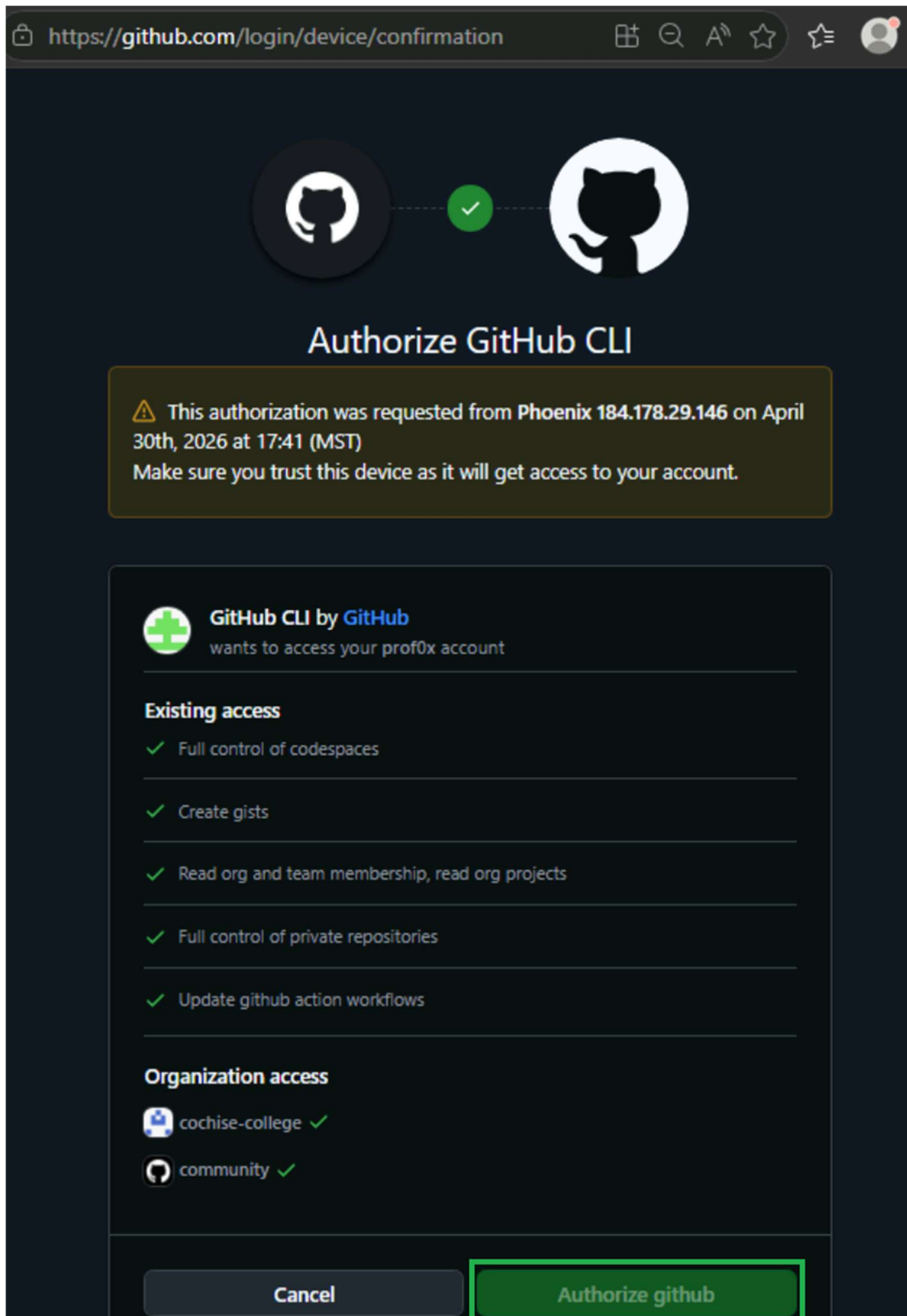

- You should already be logged in from a previous step, but if for whatever reason you're not, login to github again. If you were already logged in you should see the following screen and click "Continue"



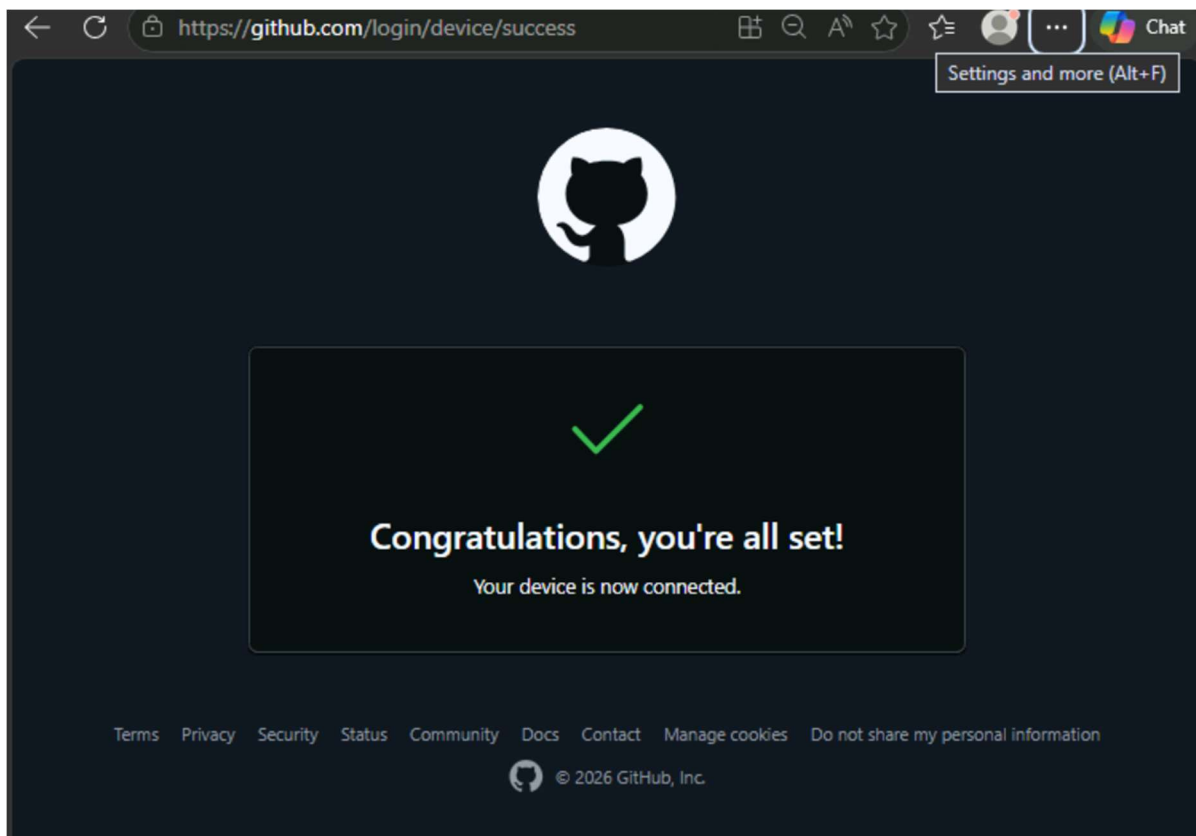
- Paste the code that you copied from the Command Prompt into the form and click "Continue"



- You will be asked to Authorize Github CLI to access your github account. Scroll down and click on “Authorize github”.



- If everything went smoothly then you should see the following screen and you can now close your browser tab.



- In your terminal, you should also see confirmation that you are logged into github in the CLI.

```
Administrator: Command Prompt

C:\Users\Student\Source>gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

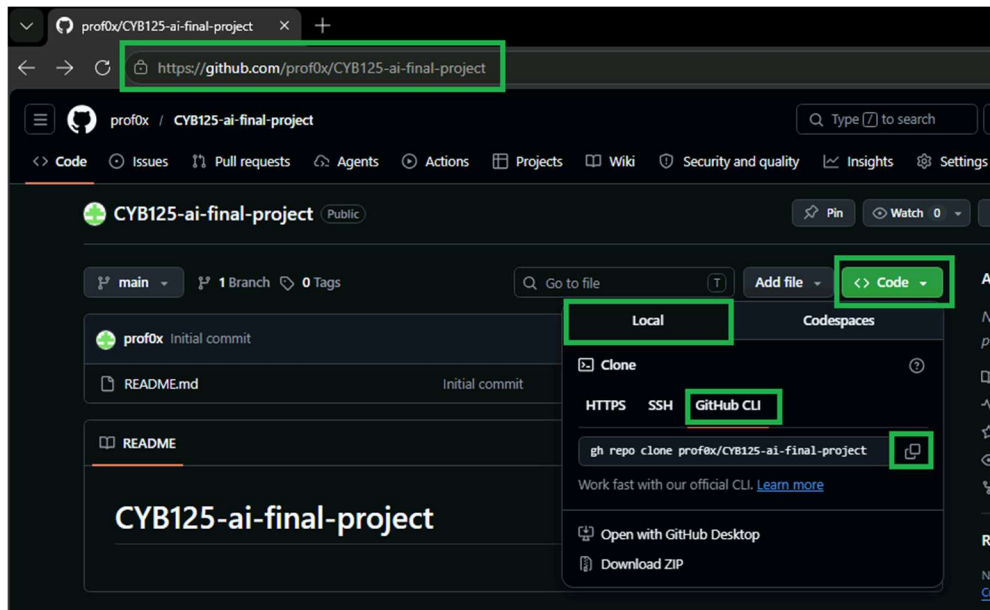
! First copy your one-time code: 1911-D90B
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as prof0x ✓

C:\Users\Student\Source>
```

You are now logged into github in the terminal. Next you will use your access to github to clone the repository that you created named **CYB125-ai-final-project**.

Step 1.8: Go to your repo in github (it should be open in another tab from the first step)

- click on the “Code Button”
- click on the “Local” tab
- click on “GitHub CLI”
- and click the square icon next to the command to copy it.
`gh repo clone <your_username>/CYB125-ai-final-project`
- Go back to the command line and paste it into your command prompt.



```
Administrator: Command Prompt
C:\Users\Student\Source>gh repo clone prof0x/CYB125-ai-final-project
Cloning into 'CYB125-ai-final-project' ...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

C:\Users\Student\Source>cd CYB125-ai-final-project
C:\Users\Student\Source\CYB125-ai-final-project>dir
Volume in drive C has no label.
Volume Serial Number is 90AB-8E3B

Directory of C:\Users\Student\Source\CYB125-ai-final-project

05/04/2026  06:46 AM    <DIR>          .
05/04/2026  06:46 AM    <DIR>          ..
05/04/2026  06:46 AM                25 README.md
               1 File(s)                25 bytes
               2 Dir(s) 19,363,069,952 bytes free

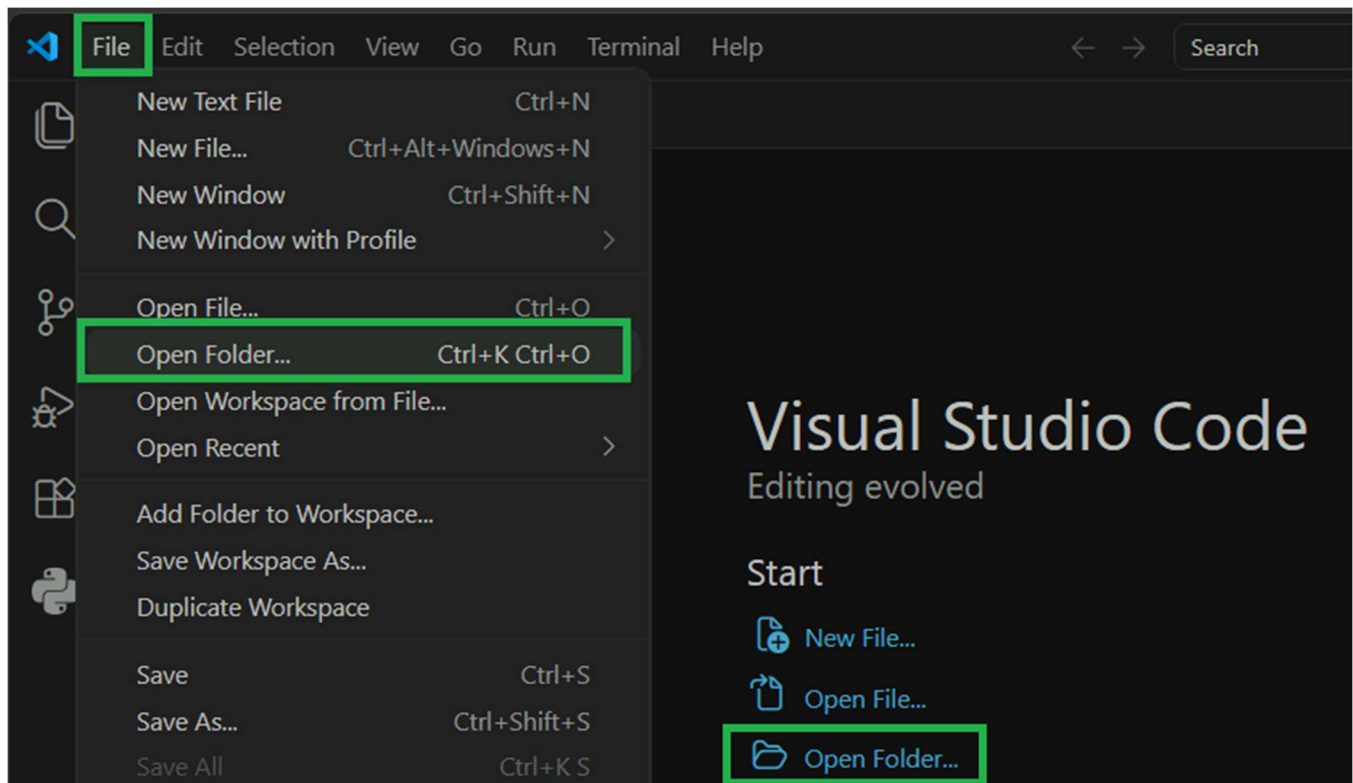
C:\Users\Student\Source\CYB125-ai-final-project>
```

- type the command “cd CYB125-ai-final-project” to change into that directory.

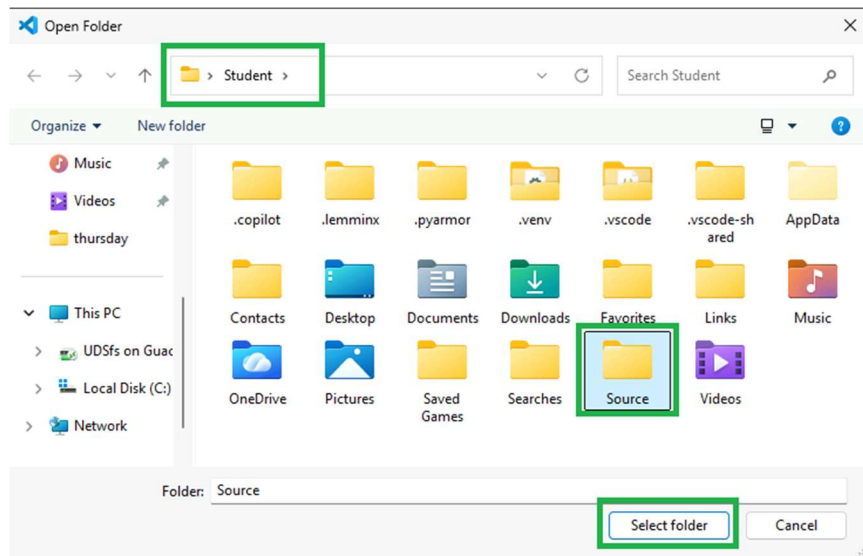
- type the command “dir” to list all contents of the directory and confirm “README.md” is there.

Step 1.9: Inside your VLE, on the Desktop there should be an Icon for Visual Studio Code.

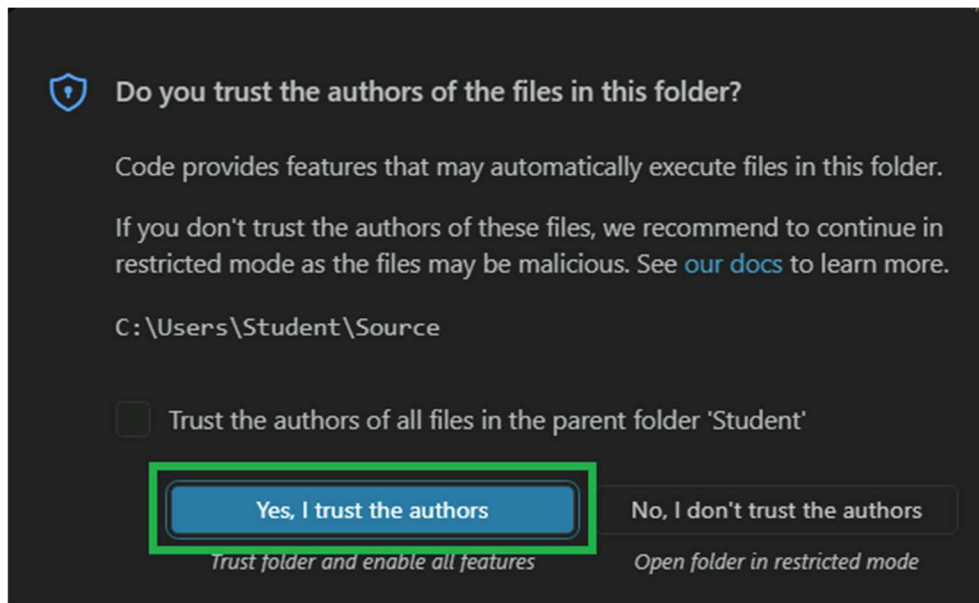
- Double click the icon on the desktop to open Visual Studio Code
 - This project has to be run on the Desktop version of Visual Studio Code, not in a codespace. This project will not run correctly in a browser-based Github Codespace.
- When VS Code opens
 - Click the “File” menu
 - Click “Open Folder”



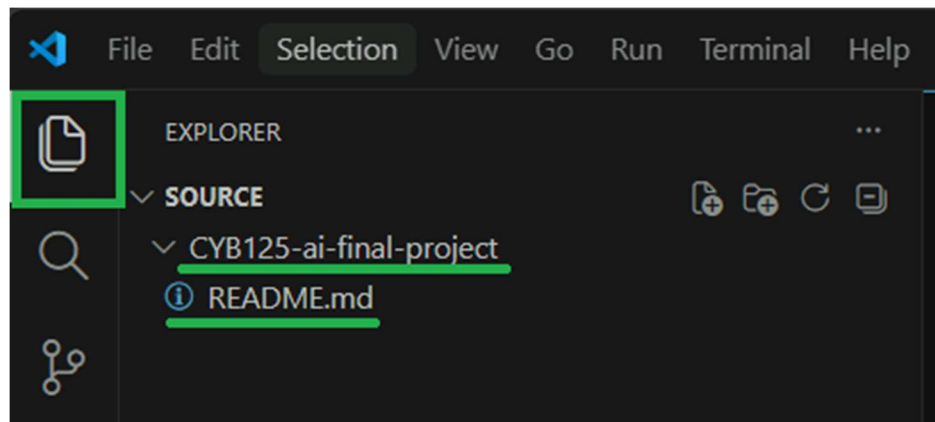
- The Open Folder dialogue should open up to the “Student” folder.
 - Click the “Source” folder that you created in a previous step.
 - Click “Select Folder”



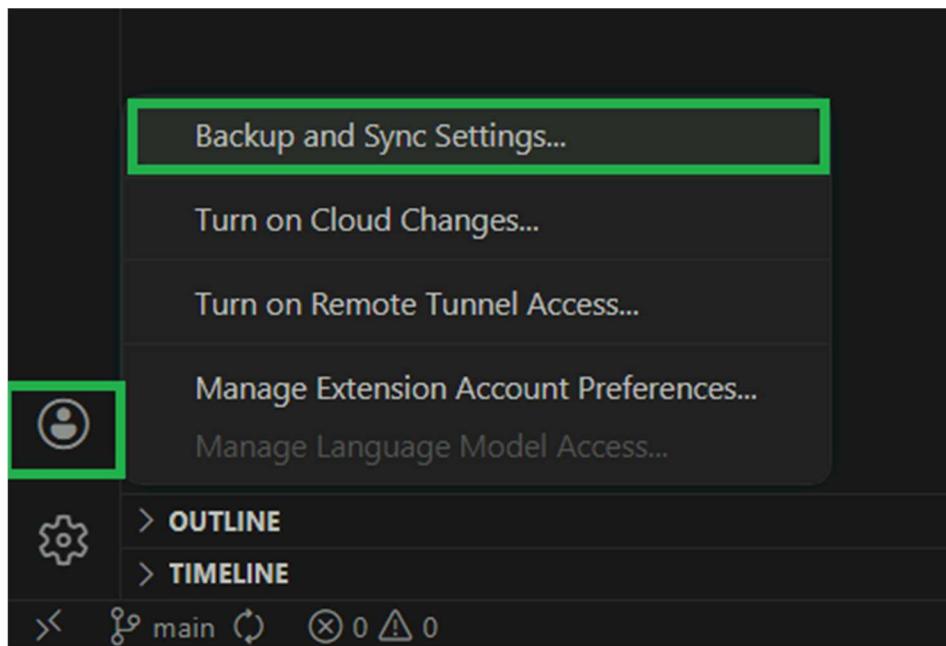
- Click “Yes I trust the authors” on the next popup.



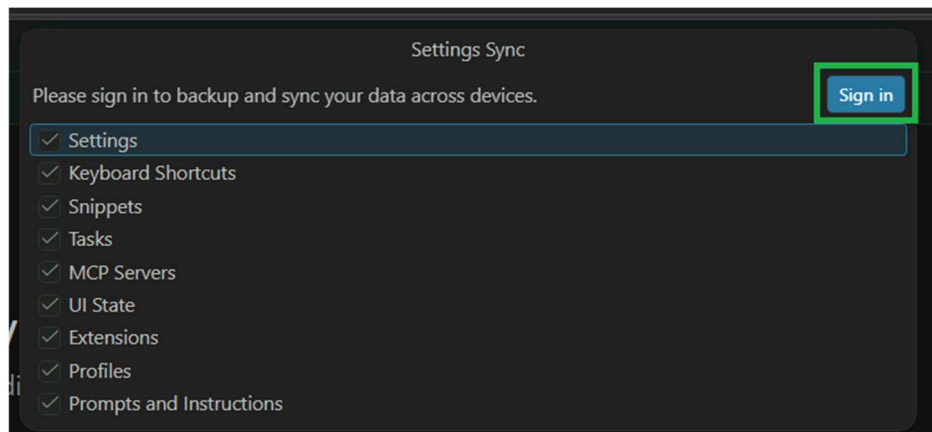
- You should see the “CYB125-ai-final-project” folder in the File Explorer.
 - If not click on the “Pages” icon to reveal the File Explorer.
 - Expand the CYB125-ai-final-project folder
 - Confirm README.md is present in the folder.
 - This folder lives on your VM so any changes that you make to it will not be reflected in your repo on github until you push those changes up (in another step).



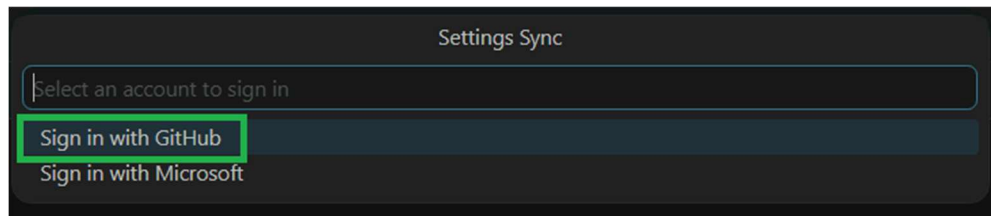
- At the bottom left corner of VS Code you should see a User Icon (a person in a circle)
 - Click on the User Icon
 - Click on “Backup and Sync Settings”



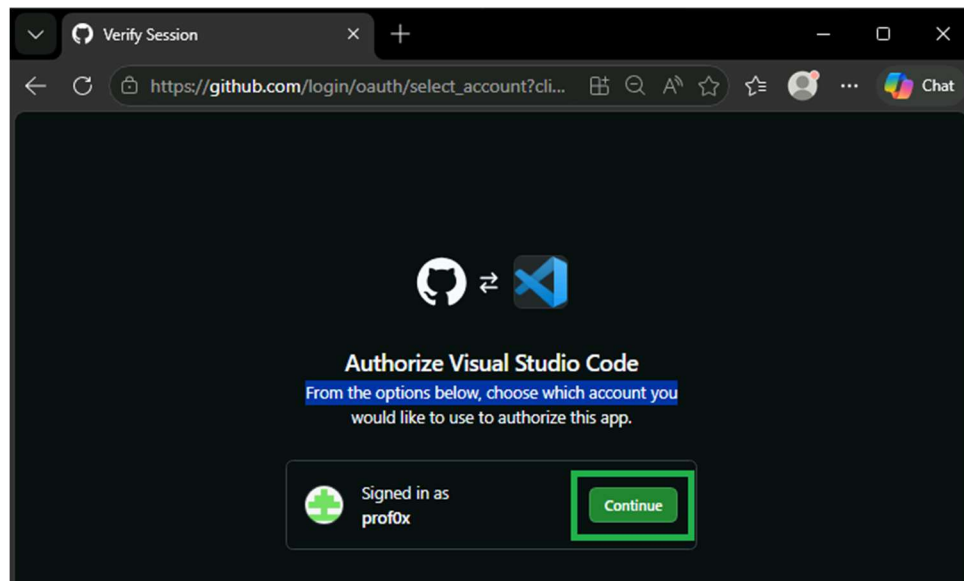
- This will cause the “Sync Settings” dialog to popup
 - Click on “Sign In”
 -



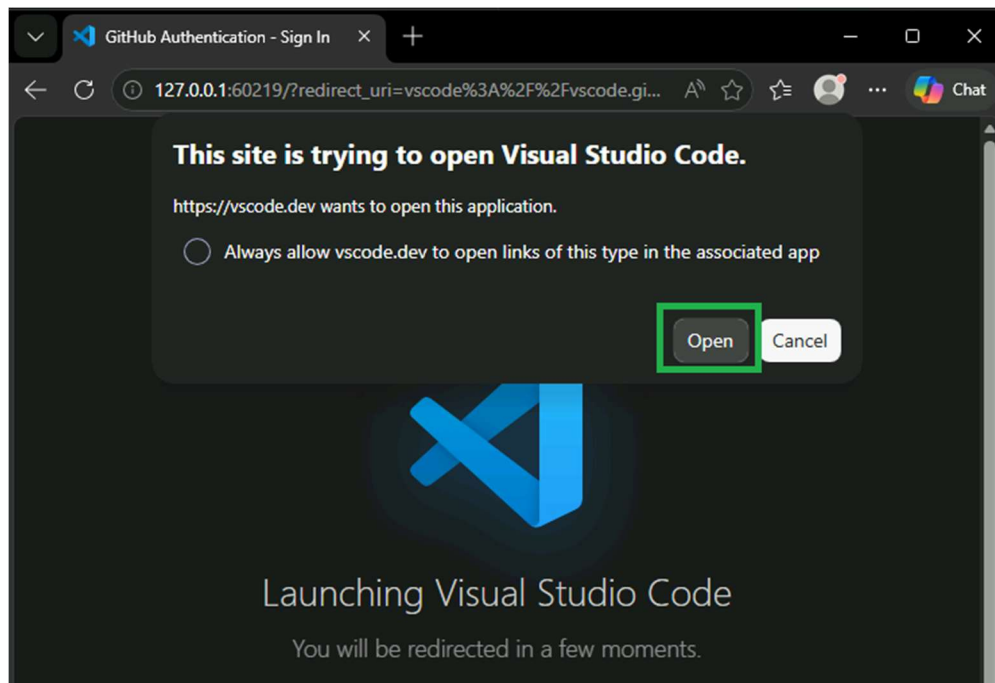
- Then click on “Sign in with Github”
- This is the last time you will have to sign into Github!



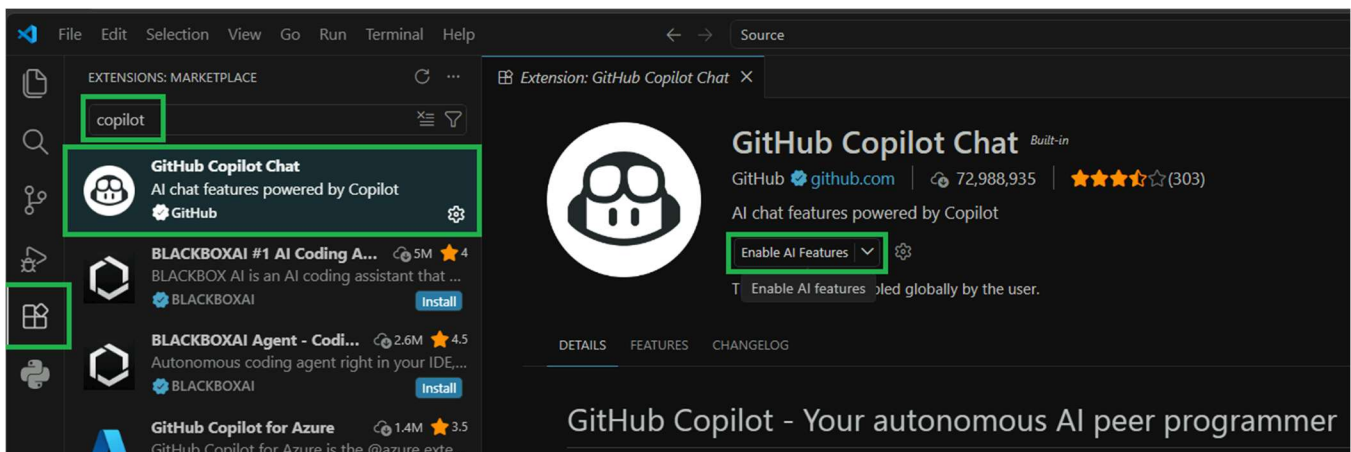
- A browser tab will popup and take you to github.com
 - You should already be signed in, if not please sign into github
 - Click “Continue” on your username

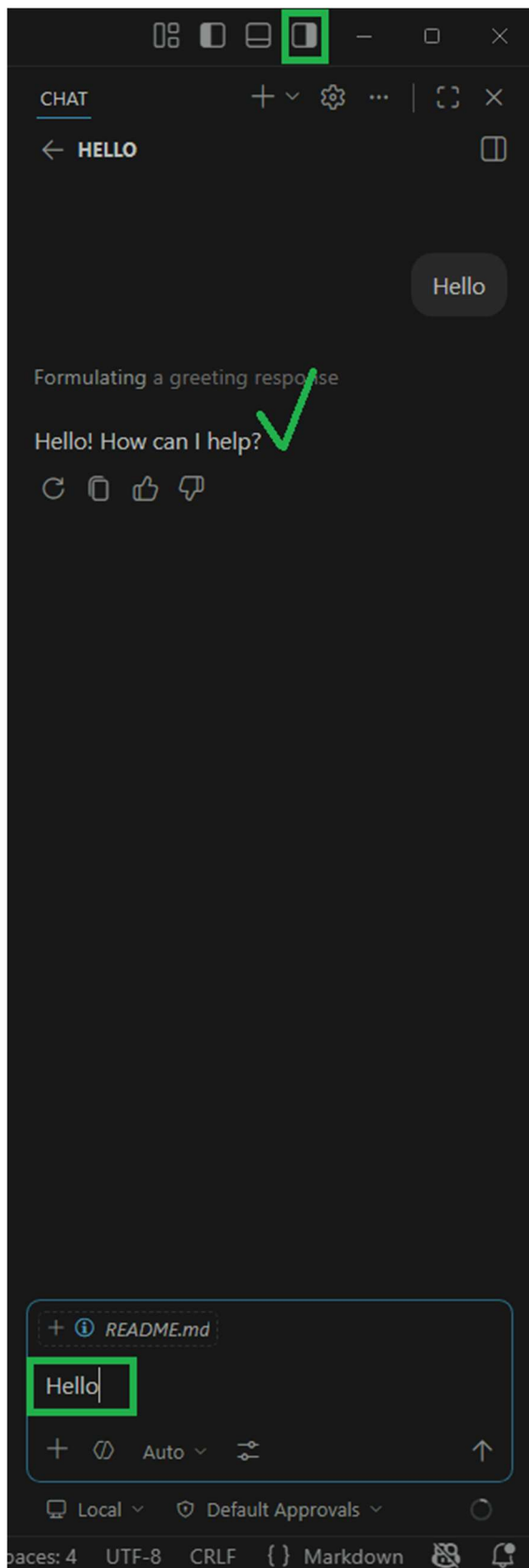


- A dialog will popup asking you to allow vscode.dev to open VS Code (Desktop)
 - Click “Open”



- Step 1.10: Last thing before we get started coding is to Enable AI features :
 - open up the extensions marketplace from the left hand navigation
 - search for “copilot”
 - Click on “GitHub Copilot Chat” and
 - Click on “Enable AI Features”
 - when the AI feature are enabled the button will say “Disable...”
 - Do not disable them, leave the AI features enabled.



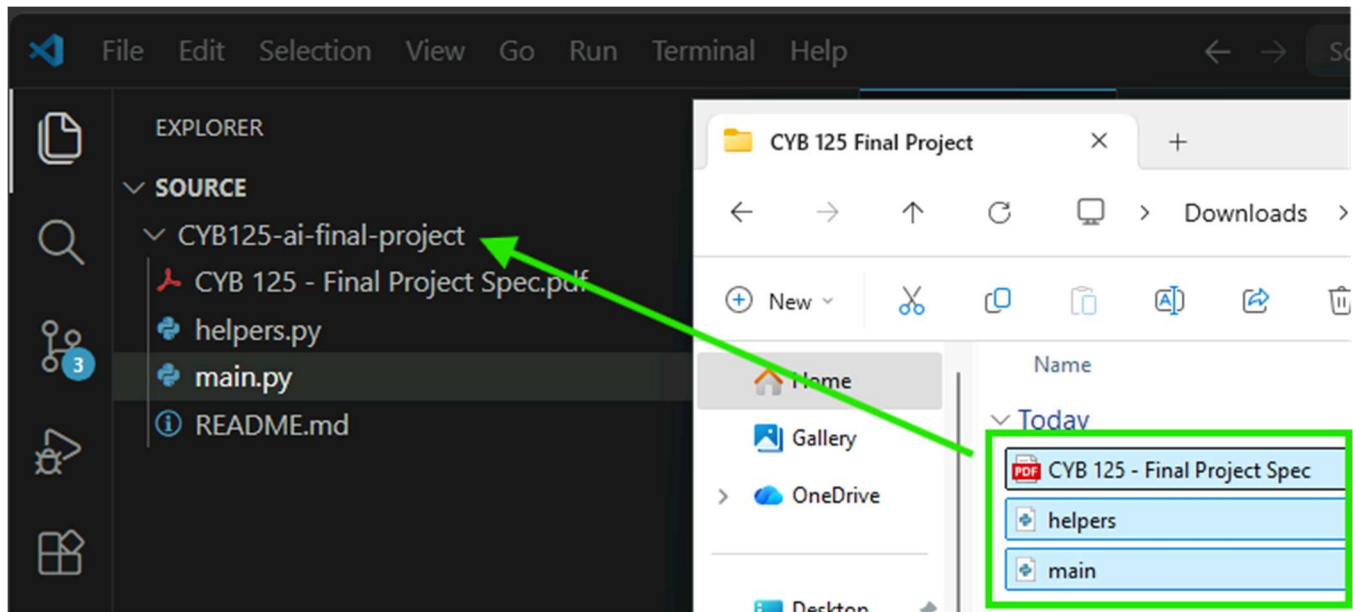


- Step 1.11: If everything went smoothly, we should be able to chat with Copilot by
 - Opening the “View” menu and
 - Click on “Chat”
 - This will open the Copilot Chat Dialog
 - If AI features are enabled you should see a chat box at the bottom right
 - Type “Hello” and press enter.
 - You can also open the Chat dialog by click on the chat icon at the top right of the screen (highlighted in the screen shot)

That completes the setup portion of the project! Just like that, with 75 easy steps, you are now ready to start coding with AI in your VLE.

Step 1.12: Download the project files from Moodle onto your VLE.

- Unzip CYB 125 Final Project.zip
- Copy the contents of the unzipped folder into your project folder



You are done with Part 0!

Part 1. Planning and Writing your README.md File (18% of your grade)

Program Behavior and Expected Output

Your program should execute in three phases:

1. **Data Input** — Your script will collect information from several sources across the OS including the Windows Registry, performance counters, and various command line utilities related to the system configuration.
2. **Data Analysis** — The majority of your analysis will go into deciding what is important enough to collect, how to store it (hint: in a dictionary object) and how to describe each piece of data with meaningful labels and groupings.
3. **Final Output** — Your program should output a .json file that contains a hierarchical list of configurations, including the source (where that configuration exists in the OS), the name of the configuration, and the current value.

Part 1. Step 0 — Open the README.md file in the VS Code Editor

Note: Please do not use any AI tools to complete this section of the project. You will be instructed when and how to use AI to help you generate responses for this project.

[Requirement 0] Your README.md file must contain all of the following headings:

- **About this Project (4-5 sentences)** – describe the script that you are planning, what it does, who it's used by, and how it's used. Again, this must be in your own words, do not copy from this project spec or use AI to generate this part. **Write it as if you were coming up with the idea and taking notes so you could remember it for yourself.**
- **Approach (3-6 sentences)** – describe the three sources of data that you will be collecting configuration data about your host VM from.
 - Windows Registry (using the winreg Python module)
 - Performance counters (sampled with the typeperf command)
 - Various Command-line utilities (you can reference the project spec for this section)
- **Data Dictionary (a Python Dictionary)** – define the data dictionary that your program will use, specifically the keys and any nested keys (but no values) You may refer to the example output file provided in the project zip. You will need to provide an empty data dictionary with all keys named descriptively.
- **Configuration Areas (1 sentence per area)** – you will need to describe each of the following areas of the OS configuration that your script will be collecting data about. They are:
 1. snapshot_metadata
 2. system_identity
 3. hardware_profile
 4. network_configuration
 5. listening_ports
 6. local_user_accounts
 7. password_policy
 8. auto_start_services
 9. running_processes
 10. installed_software
 11. installed_hotfixes
 12. persistence_locations
 13. scheduled_tasks
 14. security_posture
 15. performance_snapshot

16. network_shares

Provide a one sentence description of what each area of the OS configuration why it's important to collect as part of a system baseline. You may refer to your notes from previous classes, google, online resources but don't just copy and paste answers from AI into this section.

- **Strategy (4-5 sentences)** – Describe your strategy and approach to using AI to help you write the code for this project. Your response should explain how you plan to use AI as a learning partner rather than a code generator, what kinds of questions you intend to ask it (and what kinds you will not ask it), and how you will verify that the code it produces is correct. Reference at least one specific milestone where you expect AI assistance to be most useful and one where you plan to rely on it least.
- **Milestones** (Copy and paste this exactly into your Readme.md file) –

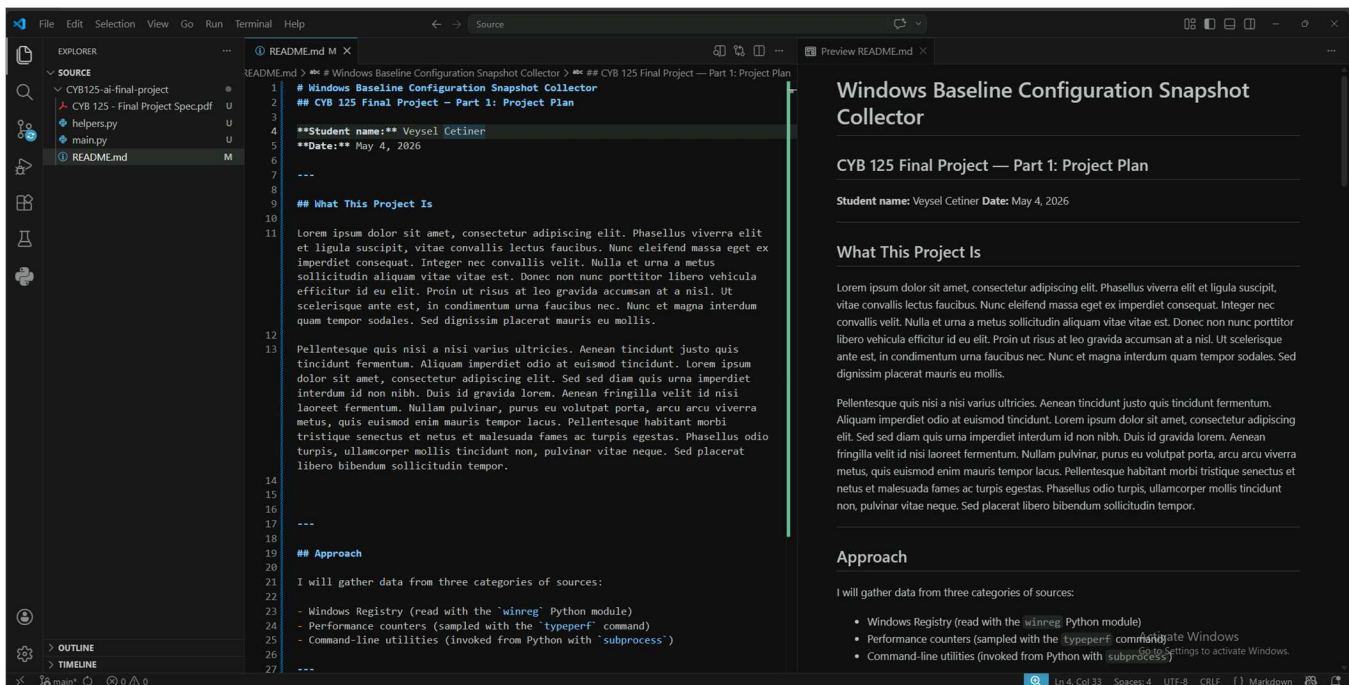
The project is structured around eight milestones, each one designed to produce a working JSON file with an additional section implemented. The milestone structure isn't just a grading convenience, **it's a deliberate AI-collaboration pattern.**

When students use AI well, they treat it like a pair programmer: they bring it small, well-scoped problems, ask it to explain things rather than just produce things, and verify its answers against an authoritative source (the textbook, the official Python docs, or their own running code). The output is code they understand and could rewrite from scratch.

The eight-milestone structure exists to force the second pattern. Each milestone is small enough that you can hold the whole thing in your head. Each milestone has a specific Python concept attached to it, so you know what you're supposed to be learning. Each milestone has a suggested AI prompt that asks for explanation, not code.

Your goal is not to finish the project as fast as possible. Your goal is to finish the project understanding what you built. Those are different goals. The milestone structure pushes you toward the second one.

[END README.MD FILE]



Note: Your README.md file uses the Markdown format to differentiate text styles like headings and lists. You don't need to do anything fancy but, at a minimum, please use the #, ##, and – format specifiers to organize the required sections of your README.md file as demonstrated above.

If you would like to explore the different Markdown styles you can read the documentation here:

<https://www.markdownguide.org/basic-syntax/>

The screenshot above shows a README.md file in raw format (left) and rendered in Preview mode (right) the way that it will appear on the landing page of your repository after you submit your project. To view your README.md file in Preview mode, right-click on README.md in the File Explorer and select “Open Preview.”

If your browser asks you to allow copy/paste from the clipboard you can select “Allow.” You can also view any .md file's preview by pressing Ctrl + Shift+ V while editing the file. You will need to drag the new tab to the right side of the screen to get the split panel view in the screenshot above.

[Requirement #1] – Once you have the questions outlined in the previous step you will submit your README.md by entering the following commands in your terminal and emailing me a copy of your repository url.

- First, open your terminal again by pressing the CTRL + J key combination. You should see a command prompt inside of VS Code.
- Copy and paste the following commands:
 - `git add README.md`
 - This command will add your README.md file to your submission
 - `git commit -m "Draft README.md Submission"`
 - This command will package your submission and relate all the files that have changed to a single entity called a “commit” in this case it's just one file.
 - `git push origin main`
 - This command will publish your commit so that I can review it.

Your terminal should look something like this when all is said and done:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Student\Source\CYB125-ai-final-project>git add README.md

C:\Users\Student\Source\CYB125-ai-final-project>git commit -m "Draft README.md Submission"
[main d664ee7] Draft README.md Submission
1 file changed, 93 insertions(+), 1 deletion(-)

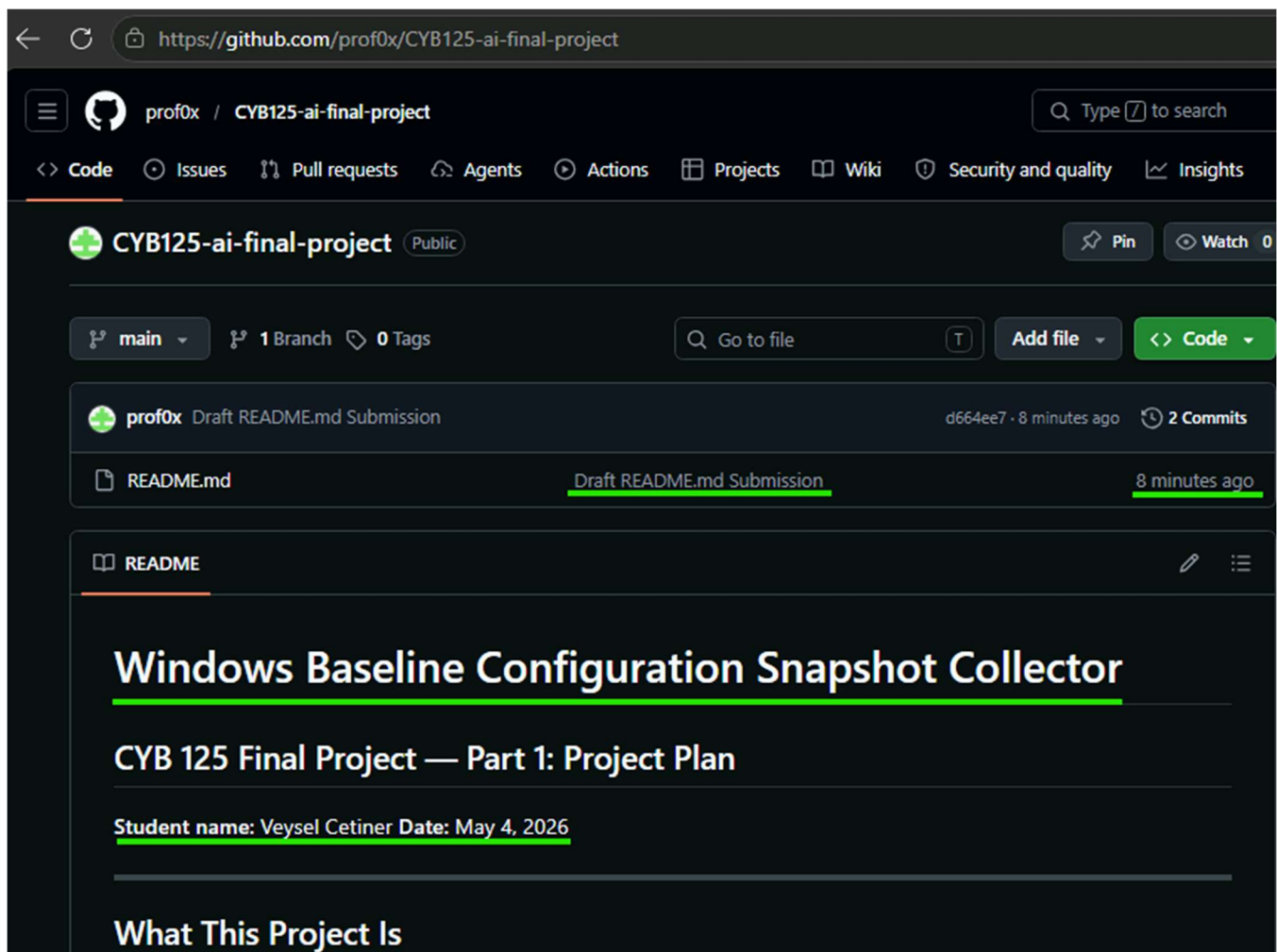
C:\Users\Student\Source\CYB125-ai-final-project>git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 3.19 KiB | 3.19 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/prof0x/CYB125-ai-final-project.git
929d5f5..d664ee7  main -> main

C:\Users\Student\Source\CYB125-ai-final-project>
```

Confirm that your README.md was properly submitted go to the landing page for the repository that you created in a previous step. Hint: it should still be open in another tab.

If you ran the commands in the previous step correctly, you should see the changes that you made to your README.md file right there on the landing page.

If not please contact me and arrange a zoom meeting to troubleshoot with me.



You are done with Part 1! **Please email me a copy of your repository URL.**

Part 2. Generating Code with AI the Right Way (63% of your grade)

Using AI to generate code is a powerful tool, but it can help or hurt you depending on how you use it while you are learning a new topic as a beginner. If you rely too much on AI without understanding what it produces, you may actually end up spending more time fixing errors, debugging unexpected behavior, and trying to understand code you didn't write yourself. However, when used wisely—as a helper, not a crutch—AI can speed up your work, teach you new patterns, and make difficult problems easier to understand.

In this assignment, the goal is to learn how to use AI as a partner while still keeping control of your own code and decisions. In part 1 you defined a program that you will now be using AI to generate the code that implements your program.

Part 2. Step 0 — Give the AI context and instruct it to read your README.md file

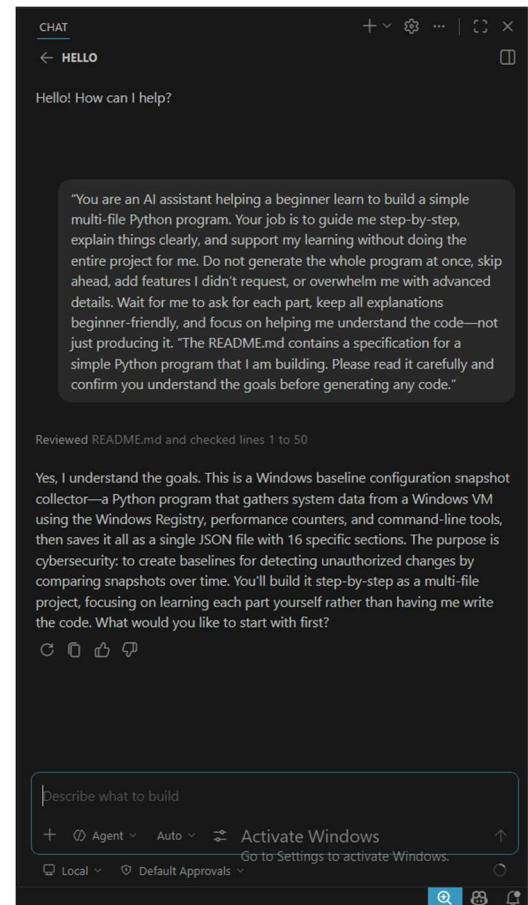
This is your very first interaction with an AI tool to help you generate content for your project. The first thing you will want to do before having the AI generate any code for you is to give it as much context as possible.

[Requirement #2] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *“You are an AI assistant helping a beginner learn to build a simple multi-file Python program. Your job is to guide me step-by-step, explain things clearly, and support my learning without doing the entire project for me. Do not generate the whole program at once, skip ahead, add features I didn’t request, or overwhelm me with advanced details. Wait for me to ask for each part, keep all explanations beginner-friendly, and focus on helping me understand the code—not just producing it. “The README.md contains a specification for a simple Python program that I am building. Please read it carefully and confirm you understand the goals before generating any code.”*

[Requirement #3] – Answer the following questions:

1. Explain why it’s important to give an AI model clear context before asking for help, and how this improves the quality of the responses you receive.
2. How did creating your own project specification improve your understanding of the program’s purpose and structure, and why it is valuable to develop your ideas independently before asking an AI to generate any code?



[Requirement #4] – Once you confirm that the code generates the correct output:.

- In your command prompt, in your project folder, copy and paste the following commands:

```
git add README.md
```

- This command will add your README.md file to your submission

```
git commit -m ""
```

- This command will package your submission and relate all the files that have changed to a single entity called a “commit” in this case it’s just one file.

```
git push origin main
```

- This command will publish your commit so that I can review it.

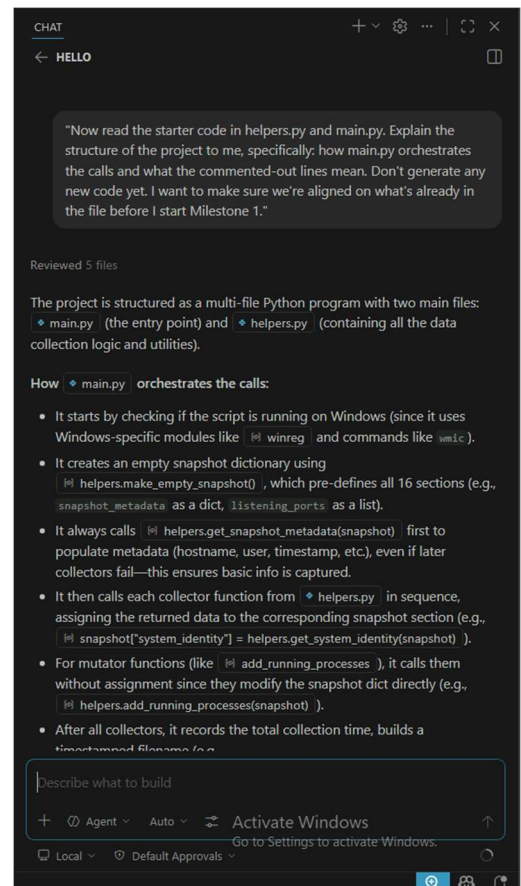
Part 2. Step 1 – Instruct the AI to read the starter code and explain its structure to you.

[Requirement #5] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"Now read the starter code in helpers.py and main.py. Explain the structure of the project to me, specifically: how main.py orchestrates the calls and what the commented-out lines mean. Don't generate any new code yet. I want to make sure we're aligned on what's already in the file before I start Milestone 1."*

[Requirement #6] – Answer the following questions:

1. The AI said main.py starts by checking if the script is running on Windows. Why does this matter for this specific project?,
2. The AI explained that you'll uncomment lines in main.py one milestone at a time. Right now, only the Milestone 2 line is active. Which sections of the JSON do you expect to be populated?



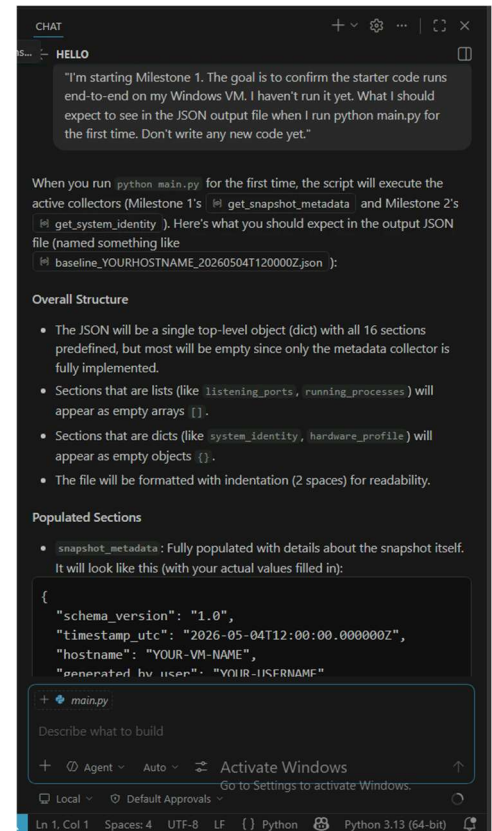
Part 2. Step 2 – Milestone #1: Metadata

[Requirement #7] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

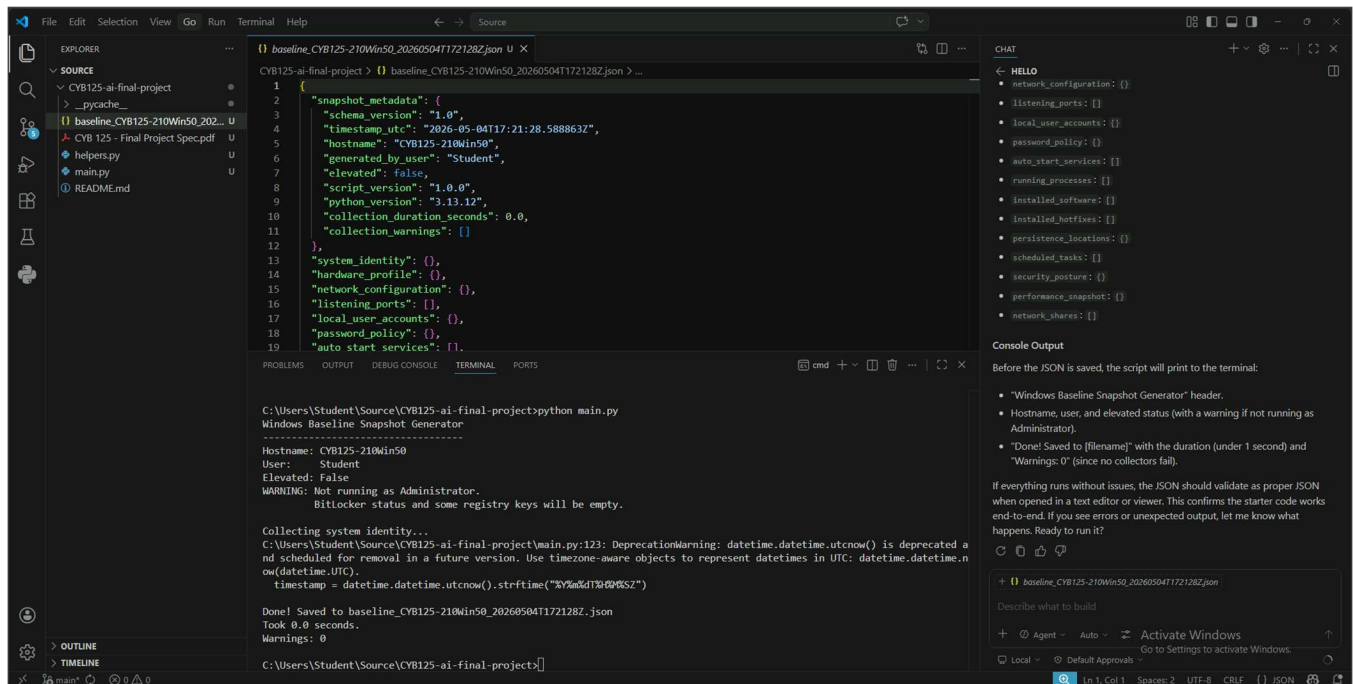
Prompt (copy and paste this): *"I'm starting Milestone 1. The goal is to confirm the starter code runs end-to-end on my Windows VM. I haven't run it yet. What I should expect to see in the JSON output file when I run python main.py for the first time. Don't write any new code yet."*

[Requirement #8] – Answer the following questions:

1. Run `python main.py` on your Windows VM. Open the resulting JSON file. Compare what you see to the AI's prediction. Did the prediction match reality exactly?
2. What does the `elevated` field in your JSON say — true or false? Run the script a second time, this time from an elevated terminal (right-click Command Prompt → Run as administrator, then re-run). What's different in the second JSON file?



Your screen should look something like this after running Milestone #1.



Part 2. Step 3 – Milestone #2: System Identity

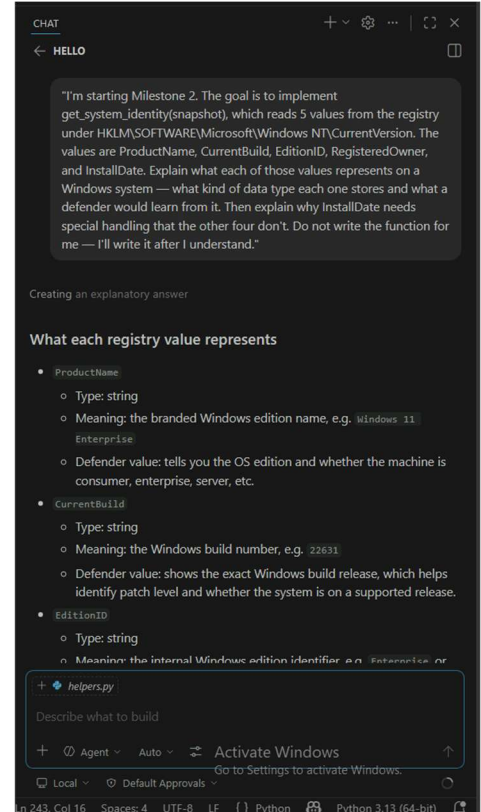
[Requirement #9] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"I'm starting Milestone 2. The goal of this milestone is to implement get_system_identity(snapshot).*

Explain what each system identity value represents on a Windows system — what kind of data type each one stores and what a defender would learn from it. Then explain why InstallDate needs special handling that the other four don't. Do not write the function for me — I'll write it after I understand."

[Requirement #10] – Answer the following questions:

1. Pick one of the five registry values that you knew the least about before reading the AI's explanation. What did you learned about it and why does that value matters for security, not just for general system identification?
2. In your own words, what is the difference between how InstallDate is stored in the registry and how it should appear in your final JSON file?



Part 2. Step 4 – Milestone #2: System Identity (Begin Code Generation):

[Requirement #11] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

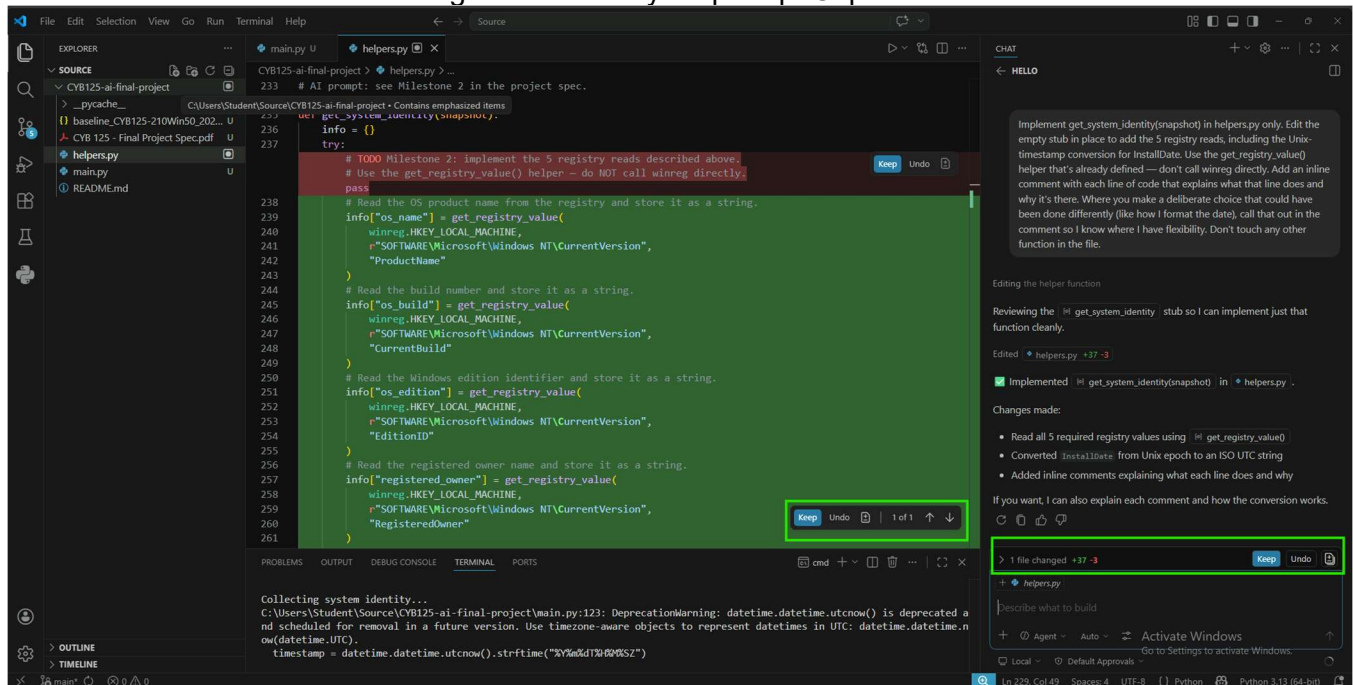
Prompt (copy and paste this): "Implement `get_system_identity(snapshot)` in `helpers.py` only. Edit the empty stub in place to add the 5 registry reads, including the Unix-timestamp conversion for `InstallDate`. Use the `get_registry_value()` helper that's already defined — don't call `winreg` directly. Add an inline comment with each line of code that explains what that line does and why it's there. Where you make a deliberate choice that could have been done differently (like how I format the date), call that out in the comment so I know where I have flexibility. Don't touch any other function in the file."

If you are happy with Copilot's changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

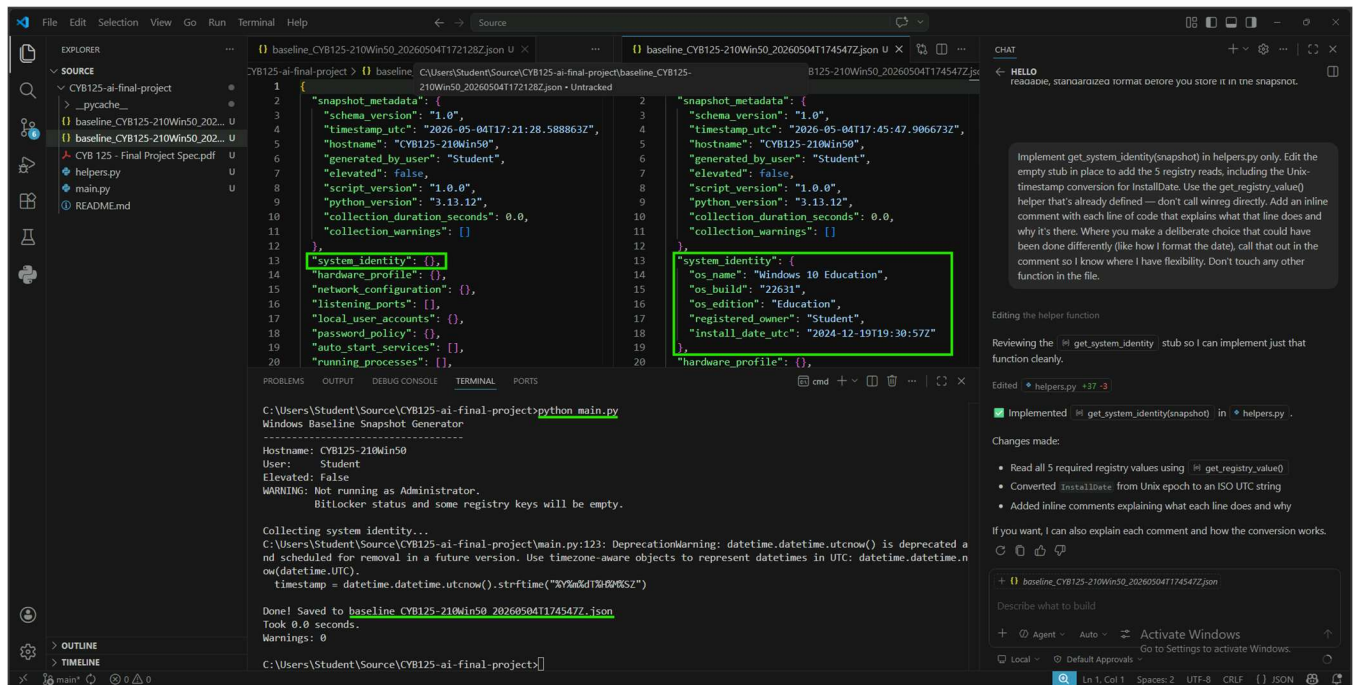
[Requirement #12] – Answer the following questions:

1. Read every comment Copilot added. Pick the one comment whose explanation taught you something you didn't already know about Python, about the registry, or about the data and explain what you learned.
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Compare what you see in the `system_identity` block to the prediction you made in your Milestone 2.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run python main.py and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

[Requirement #13] – Once you confirm that the code generates the correct output:.

- In your command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a .py extension to your submission

- `git commit -m "Milestone #1 Metadata"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

- `git push origin main`

- This command will publish your commit so that I can review it.

Part 2. Step 5 – Milestone #3 Password Policy:

[Requirement #14] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"I'm starting Milestone 3. Please uncomment the appropriate lines for milestone 3. The goal is to implement `get_password_policy(snapshot)` in `helpers.py` only. The 7 fields I need are listed in the `TODO` block above the function.*

Suggest a way to handle the case where the value is the word 'Never' instead of a number. (For example, on some systems 'Maximum password age' says 'Never'.) The fields should be integers in my JSON, or None if the source said Never — never the string 'Never'.

Finally, edit the empty stub in place to implement the function. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any other function in the file. Use `run_command()` to invoke `net accounts` — don't call `subprocess.run` directly."

If you are happy with Copilot's changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #15] – Answer the following questions:

1. Run the `net accounts` command at your Windows terminal yourself (don't use the Python script, type the command in directly). Does the output it match the results that Copilot's code produces?
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Confirm that both `system_identity` (from Milestone 2) and `password_policy` (from Milestone 3) are populated correctly. Did adding the new collector accidentally break anything in the previous one?

[Requirement #16] – Once you confirm that the code generates the correct output:.

- In your command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a `.py` extension to your submission

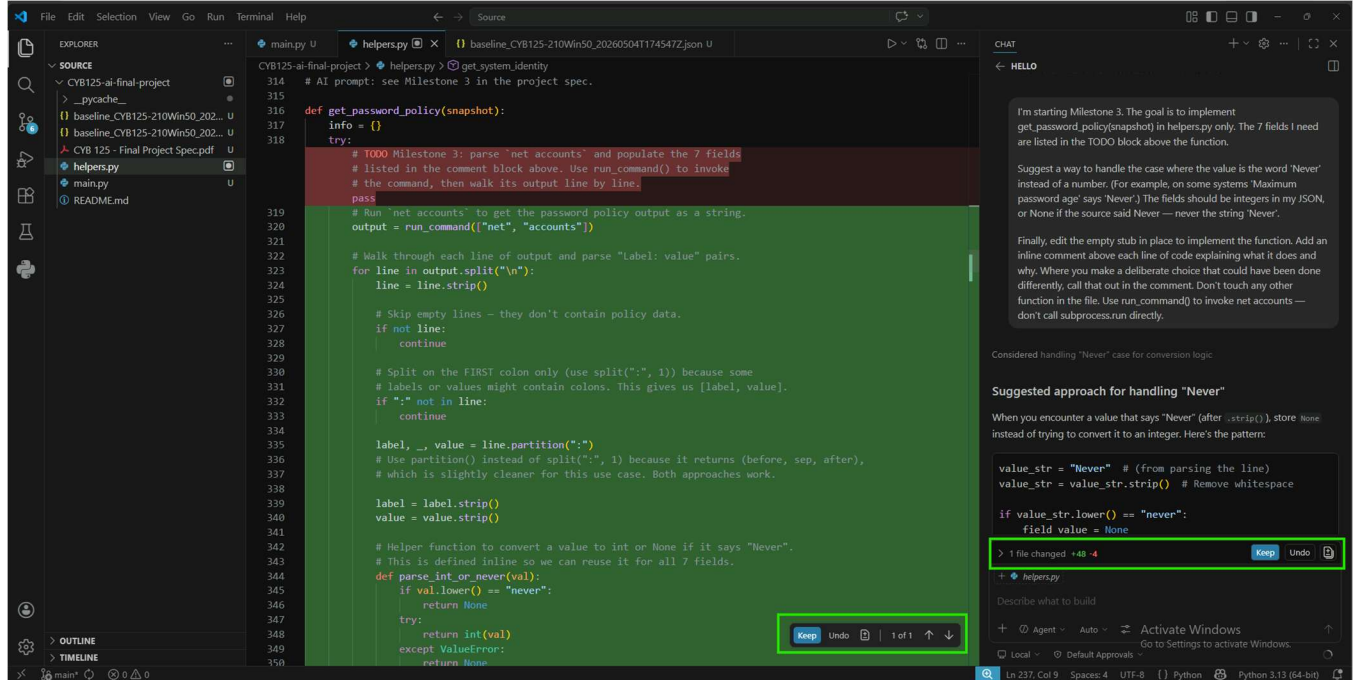
- `git commit -m "Milestone #3 Password Policy"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

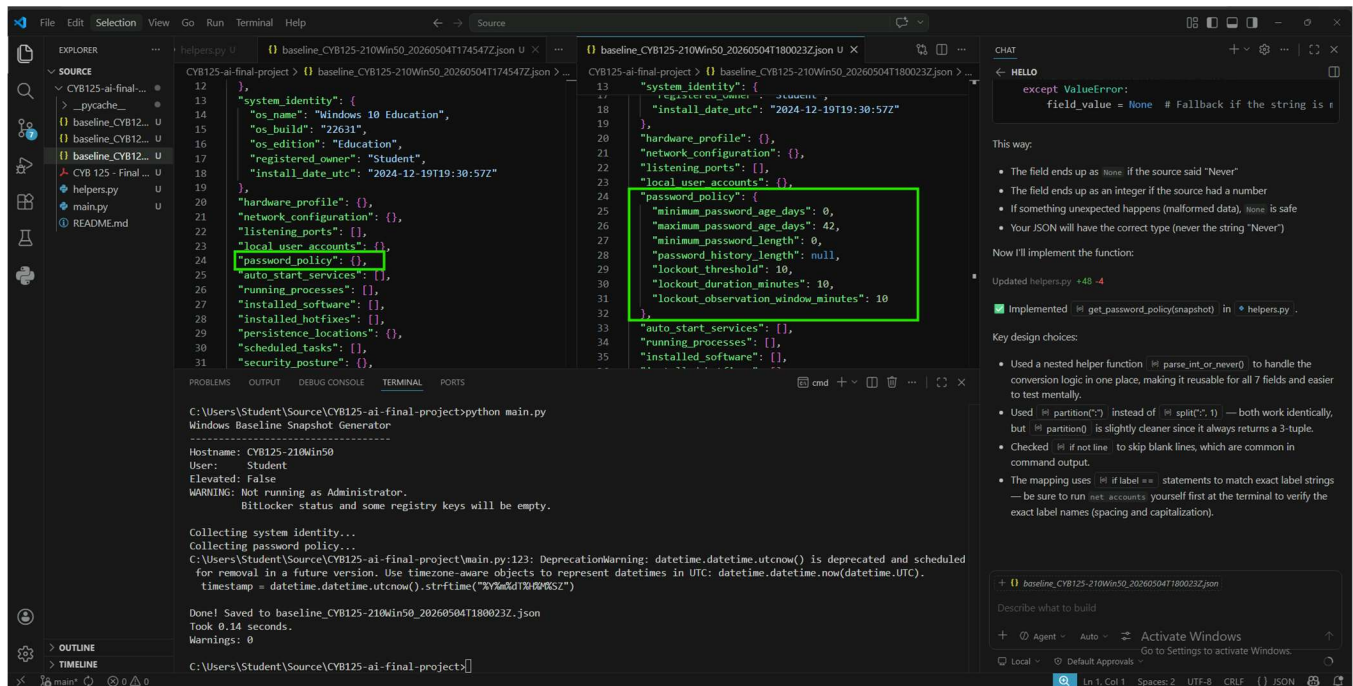
- `git push origin main`

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Part 2. Step 6 – Milestone #4 Installed Software:

[Requirement #17] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"I'm starting Milestone 4. Please uncomment the appropriate lines for milestone 4. The goal is to implement `get_installed_software(snapshot)` in `helpers.py` only. The TODO block above the function lists the 4 fields I want for each program running on the host.*

Explain in plain English how `winreg.EnumKey(key, i)` is used to walk subkeys. Specifically: what does it return, what error does it raise when there are no more subkeys, and how do I structure my loop around that? This pattern is different from a normal Python for loop and I want to understand it before I read the code.

The `InstallDate` registry value comes back as a string in YYYYMMDD format like "20250902" (different from the Unix timestamp in Milestone 2 — this one is already a string). Explain how I should convert that to an ISO date like "2025-09-02". Some programs don't have an `InstallDate` at all — what should the field show in that case?

Finally, edit the empty stub in place to implement the function. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any other function in the file. Use `get_registry_value()` for reading individual values."

If you are happy with Copilot's changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #18] – Answer the following questions:

1. In Milestone 2, `InstallDate` was a Unix timestamp (an integer). In Milestone 4, `InstallDate` is a string in YYYYMMDD format. Both are stored under registry keys with similar names, but they're encoded differently. Explain in your own words why this is confusing and what general lesson it teaches about reading data from the Windows Registry?
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Open the JSON and answer: roughly how many programs did your script find? Search the list for one program you know is installed (Chrome, VS Code, Python itself) and confirm it's there. If a program you expected isn't in the list, what's the most likely reason?
 - Hint: there's a known gap in this milestone's coverage that you'll fix later. Where are 32-bit programs registered in the Windows Registry vs. 64-bit programs.

[Requirement #19] – Once you confirm that the code generates the correct output:

- In your command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a `.py` extension to your submission

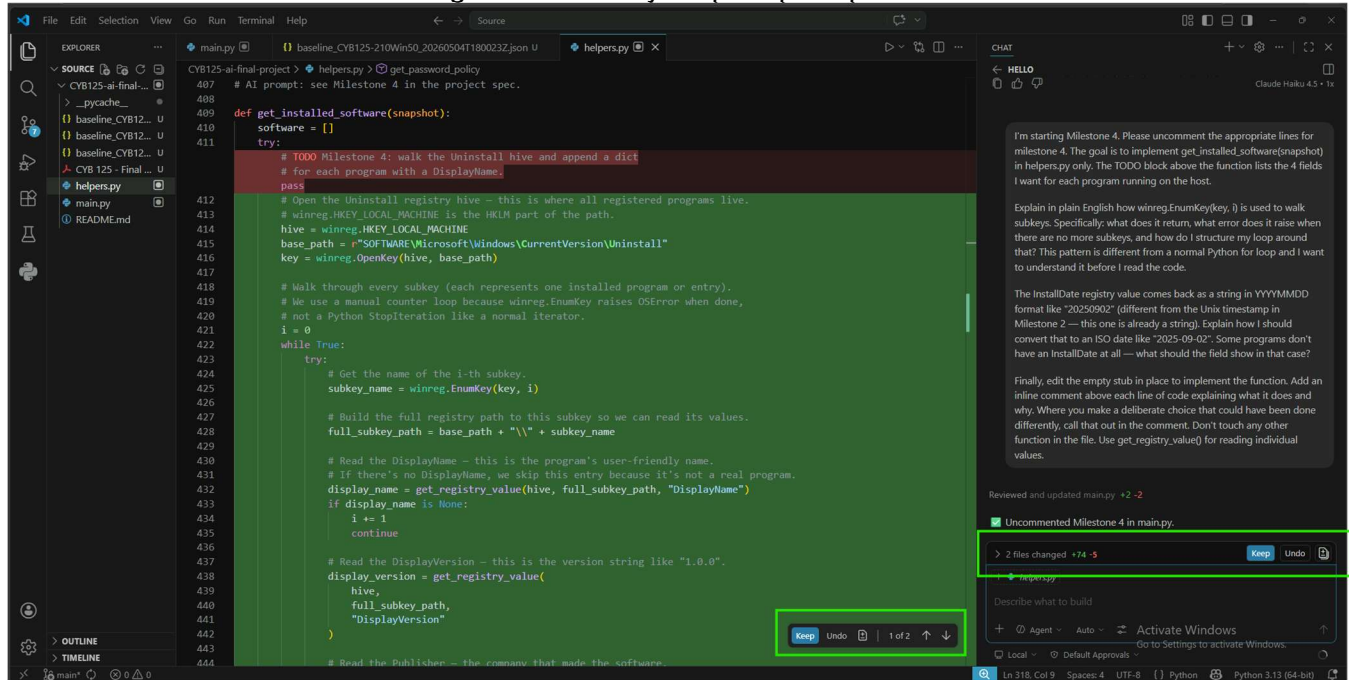
- `git commit -m "Milestone #4 Installed Software"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

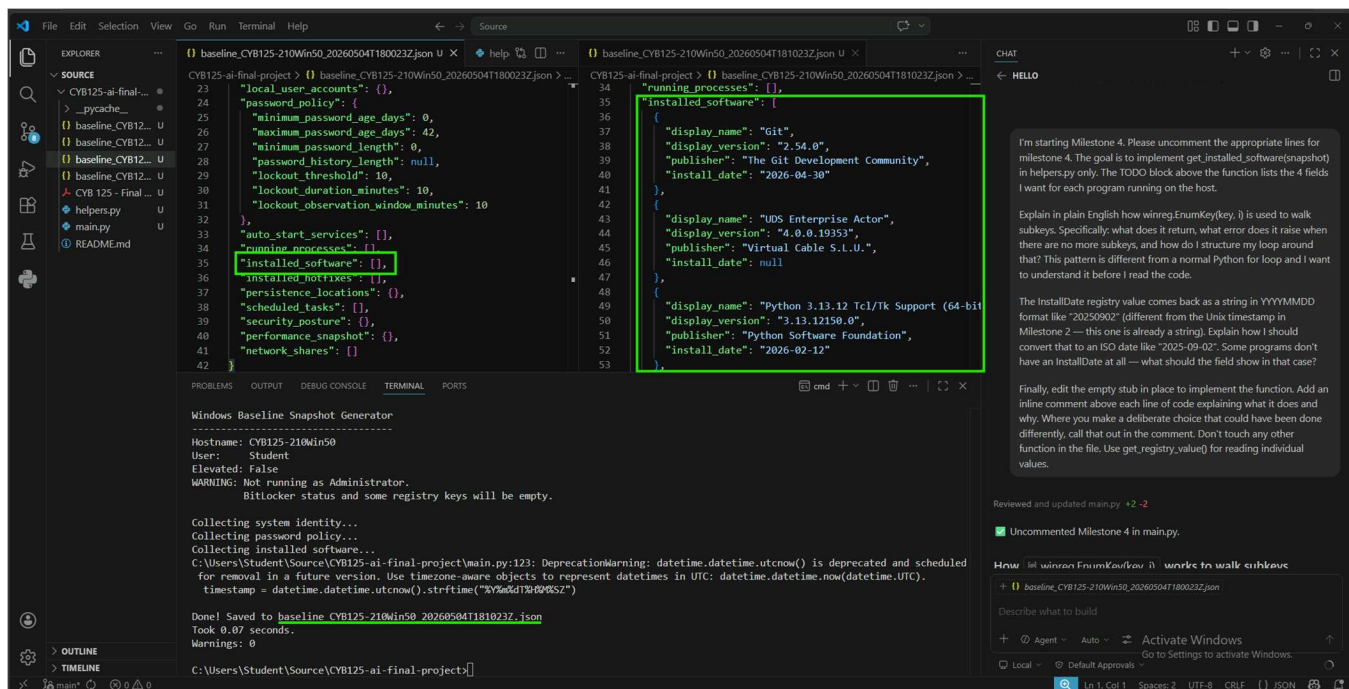
- `git push origin main`

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Part 2. Step 7 – Milestone #5 Running Processes:

[Requirement #20] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"I'm starting Milestone 5. Please uncomment the appropriate lines for milestone 5. The goal is to implement `add_running_processes(snapshot)` in `helpers.py` only. The TODO block above the function lists the 5 fields per entry.*

Notice the function name is `add_running_processes`, not `get_running_processes`. This is the rubric's required "void function that mutates an argument" pattern. The function returns `None` and modifies the `snapshot` dict directly.

Explain in plain English why this function is written as a mutator (returns `None`, modifies `snapshot` in place) instead of as a regular collector (returns a list, gets assigned in `main.py`). What does it mean that 'dictionaries are mutable' in Python, and why does that make this pattern work?

What is the output of `tasklist /fo csv` and how do you plan to parse it?

Finally, edit the empty stub in place to implement the function. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any other function in the file. Use `run_command()` to invoke `tasklist` — don't call `subprocess.run` directly. Remember: the function does not return anything — `main.py` will call it as `helpers.add_running_processes(snapshot)` with no assignment."

If you are happy with Copilot's changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #21] – Answer the following questions:

1. Imagine a student wrote `snapshot["running_processes"]` `helpers.add_running_processes(snapshot)` in `main.py` instead of just `helpers.add_running_processes(snapshot)`. What would `running_processes` end up being in the JSON file, and why? Don't actually try this, just think it through.
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Roughly how many running processes did your script find? Open Task Manager on your VM and count how many processes are running there. Are the numbers similar? If they're very different (more than 20% off), what could account for the discrepancy?

[Requirement #22] – Once you confirm that the code generates the correct output:

- In your command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a `.py` extension to your submission

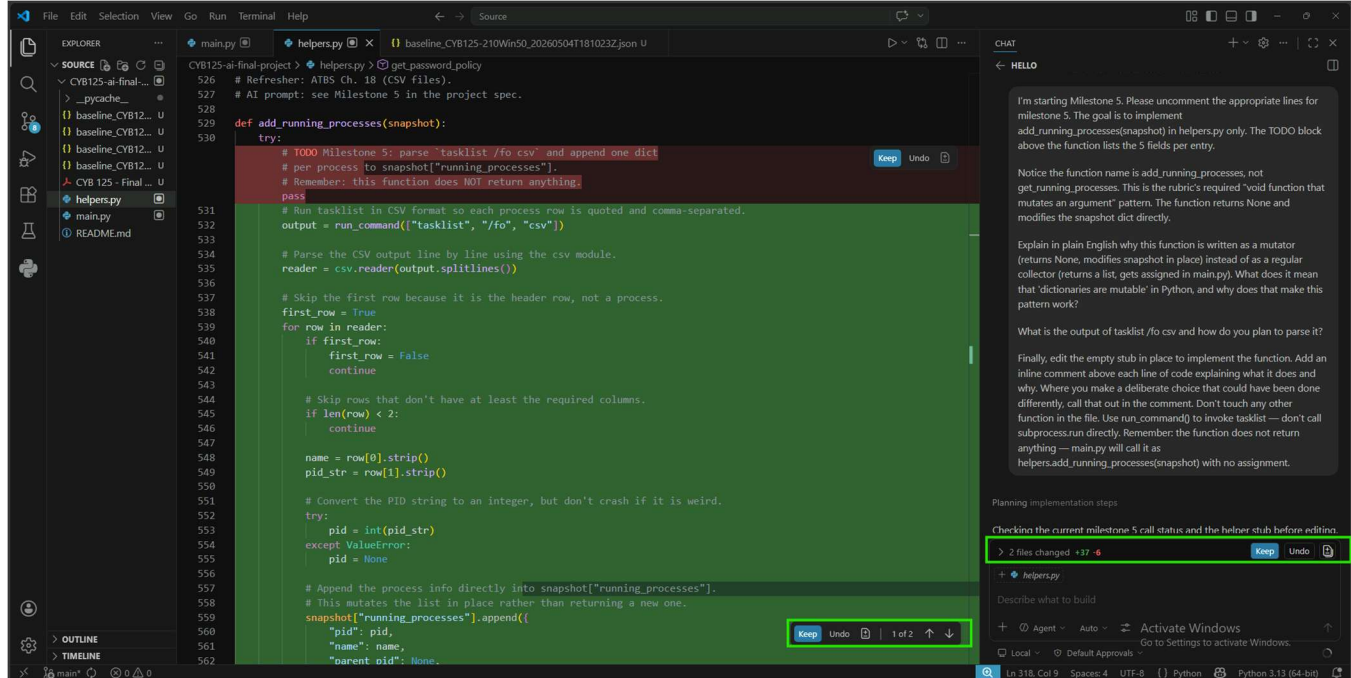
- `git commit -m "Milestone #5 Running Processes"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

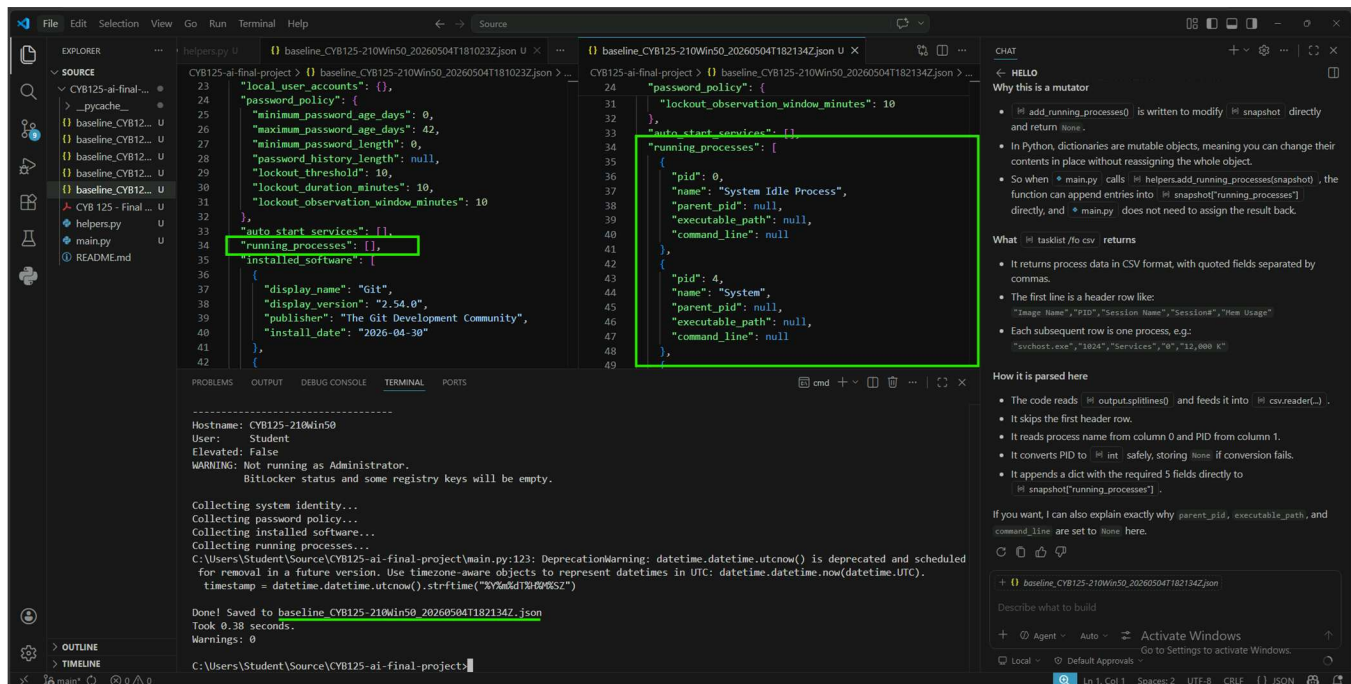
- `git push origin main`

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Note: You can stop here, skip to the Final Reflection and still get an A (90%). If you want to aim for the 100% flawless victory on this assignment, please continue.

Part 2. Step 8 – Milestone #6 Persistence Locations:

[Requirement #23] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): "I'm starting Milestone 6. Please uncomment the appropriate lines for milestone 6. The goal is to implement `get_persistence_locations(snapshot)` in `helpers.py` only. The `TODO` block above the function lists the four registry keys to capture.

The four registry keys all need the same operation: enumerate every name/value pair under the key. This is different from Milestone 4, where I walked `SUBKEYS`. Explain how `winreg.EnumValue(key, i)` differs from `winreg.EnumKey(key, i)`.

Finally, edit the empty stub in place to implement the function. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any other function in the file."

If you are happy with Copilot's changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #24] – Answer the following questions:

1. Explain why an attacker would prefer to plant a malicious entry in `HKCU\Run` rather than `HKLM\Run`, even though `HKLM` runs at every login for every user and why a defender comparing two snapshots would care about both keys equally. Why does this milestone matter more for cybersecurity than, say, the password policy milestone?
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Look at the four registry arrays in `persistence_locations`. At least one of them is probably non-empty (most likely `hkcure_run`, often containing `OneDrive` or similar). Pick one entry from any of the four arrays and answer: do you recognize the program it points to? If yes, why is it set to auto-run? If no, what would you do to investigate whether it's legitimate?

[Requirement #25] – Once you confirm that the code generates the correct output:

- In your command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a `.py` extension to your submission

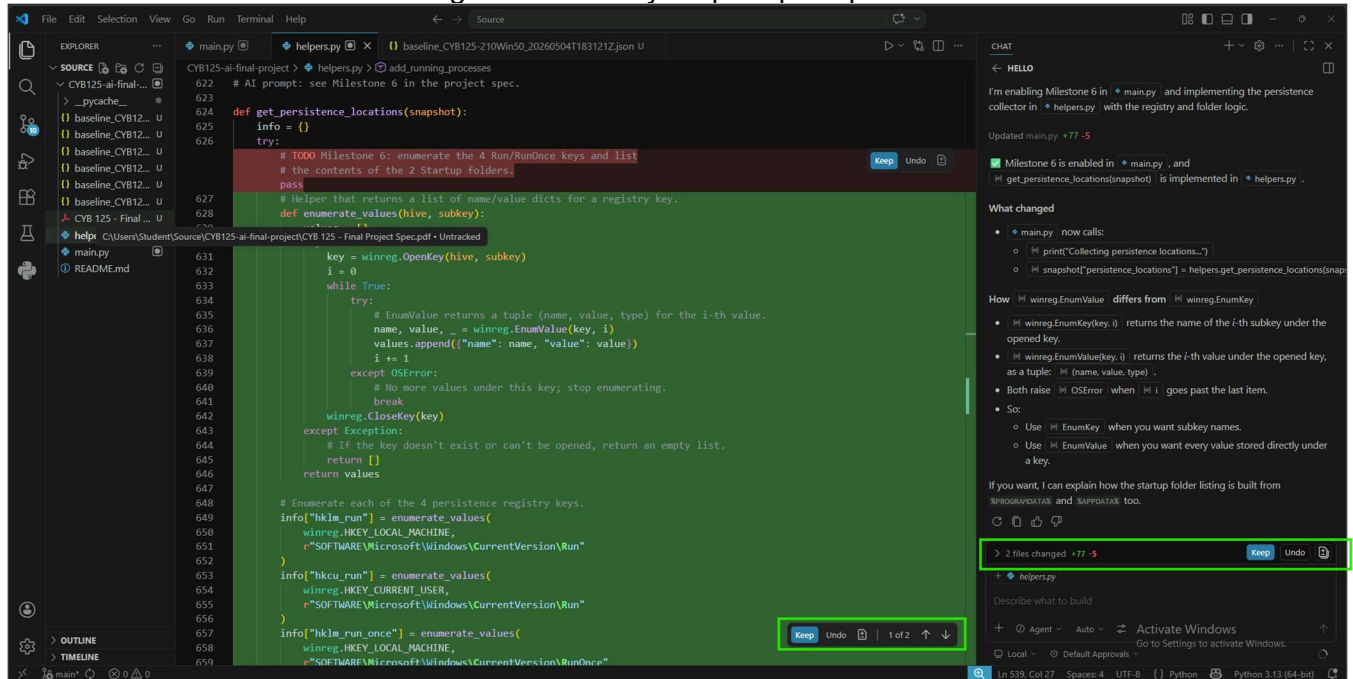
- `git commit -m "Milestone #6 Persistence Locations"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

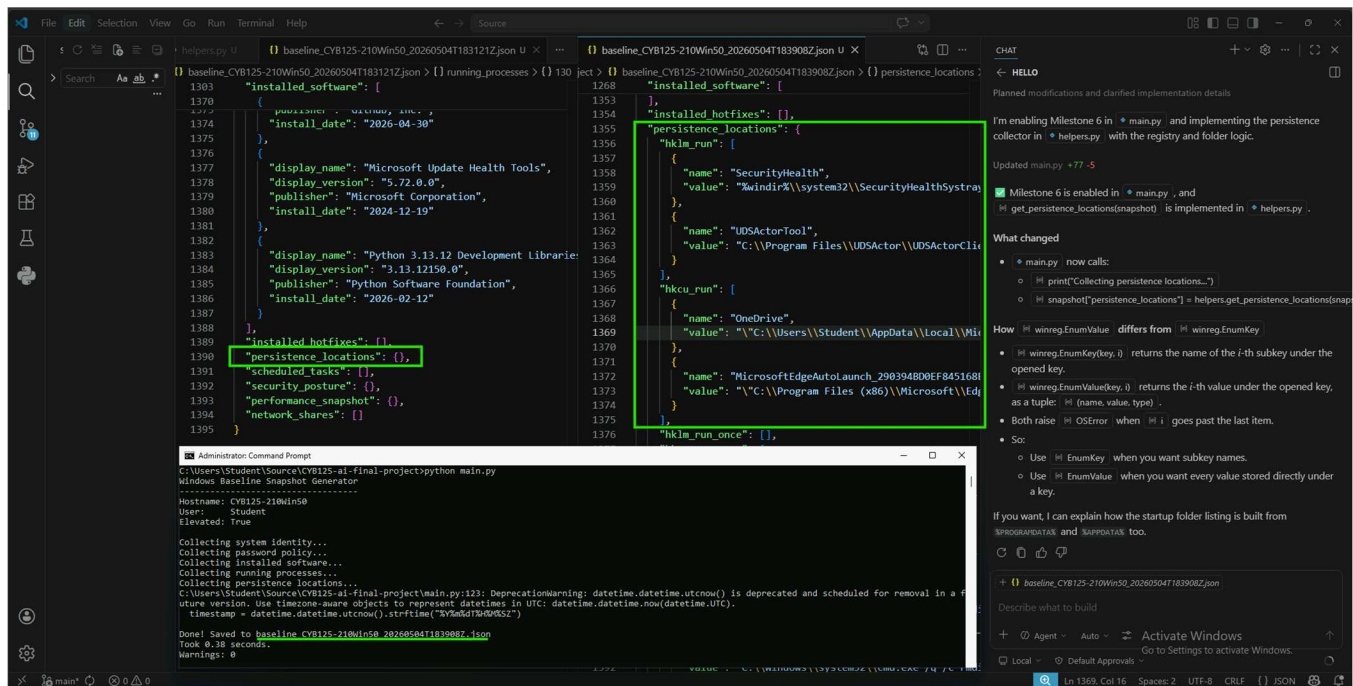
- `git push origin main`

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Part 2. Step 9 – Milestone #7 Network Configuration and Ports:

[Requirement #26] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): *"I'm starting Milestone 7. Please uncomment the appropriate lines for milestone 7. This milestone has TWO collectors — `get_network_configuration(snapshot)` and `add_listening_ports(snapshot)`. Implement both. The TODO block above each function lists the fields per adapter. Explain how you plan to implement these two collectors in relation to each other.*

Edit the empty stubs for `get_network_configuration` and `add_listening_ports` in place. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any other function in the file. Use `run_command()` to invoke `ipconfig /all`."

If you are happy with Copilots changes, click on “Keep” otherwise click “Undo” and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #27] – Answer the following questions:

1. Run `netstat -ano` directly in your Windows terminal. Look through the output and find at least one line that uses IPv6 (look for `:::` or `:::1` in the local address). Confirm that line shows up in your `listening_ports` array with the right port and PID. Then explain in 1–2 sentences why `rfind(".")` works correctly for both that IPv6 line and a normal IPv4 line.
2. Save the changes Copilot made to your `main.py`, run it in your command prompt, and open the JSON file that was generated as output. Look at `listening_ports` in your JSON. How many ports are listening? Pick one entry — preferably a non-obvious one (not 135, 445, or 5040, which are Windows defaults) — and answer: do you recognize the program in `owning_process_name`? Why does that program have a port open? If you don't recognize it, what would you do to investigate?

[Requirement #28] – Once you confirm that the code generates the correct output:.

- In you command prompt, in your project folder, copy and paste the following commands:

```
• git add *.py
```

- This command will add all files with a `.py` extension to your submission

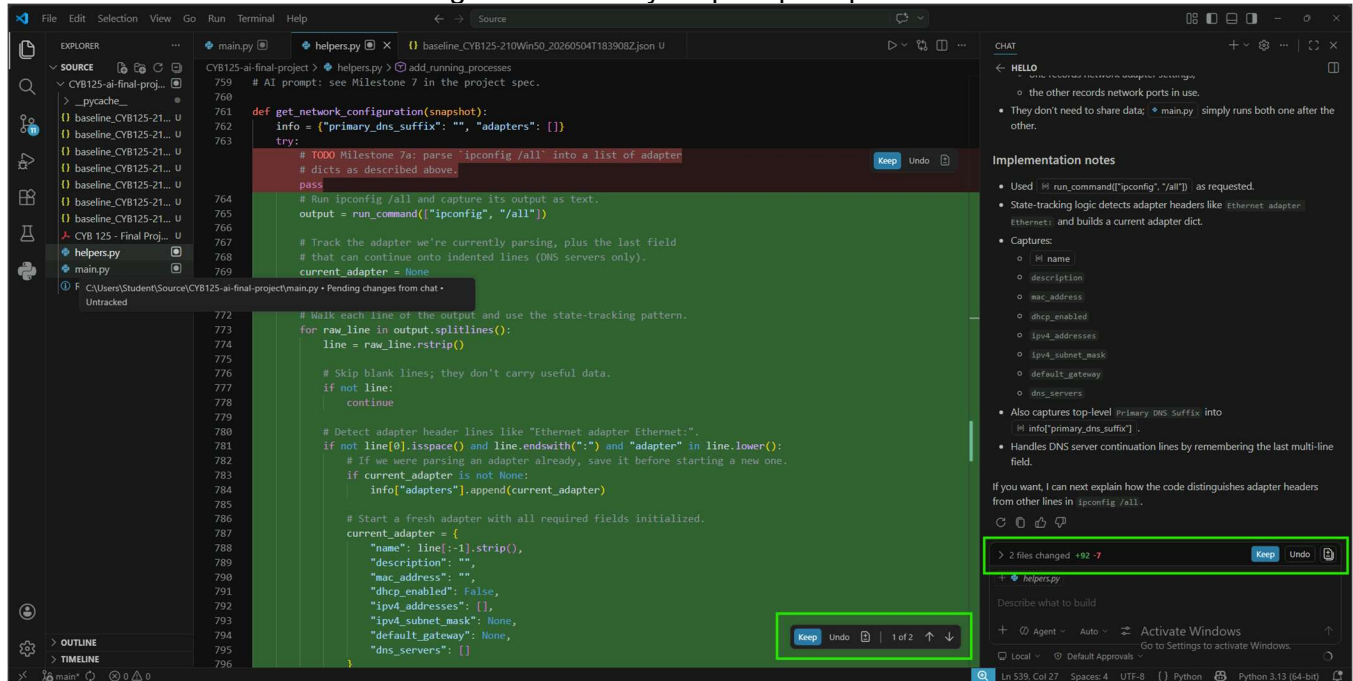
```
git commit -m "Milestone #7 Network Config and Ports"
```

- This command will package your submission and relate all the files that have changed to a single entity called a “commit” in this case it’s just one file.

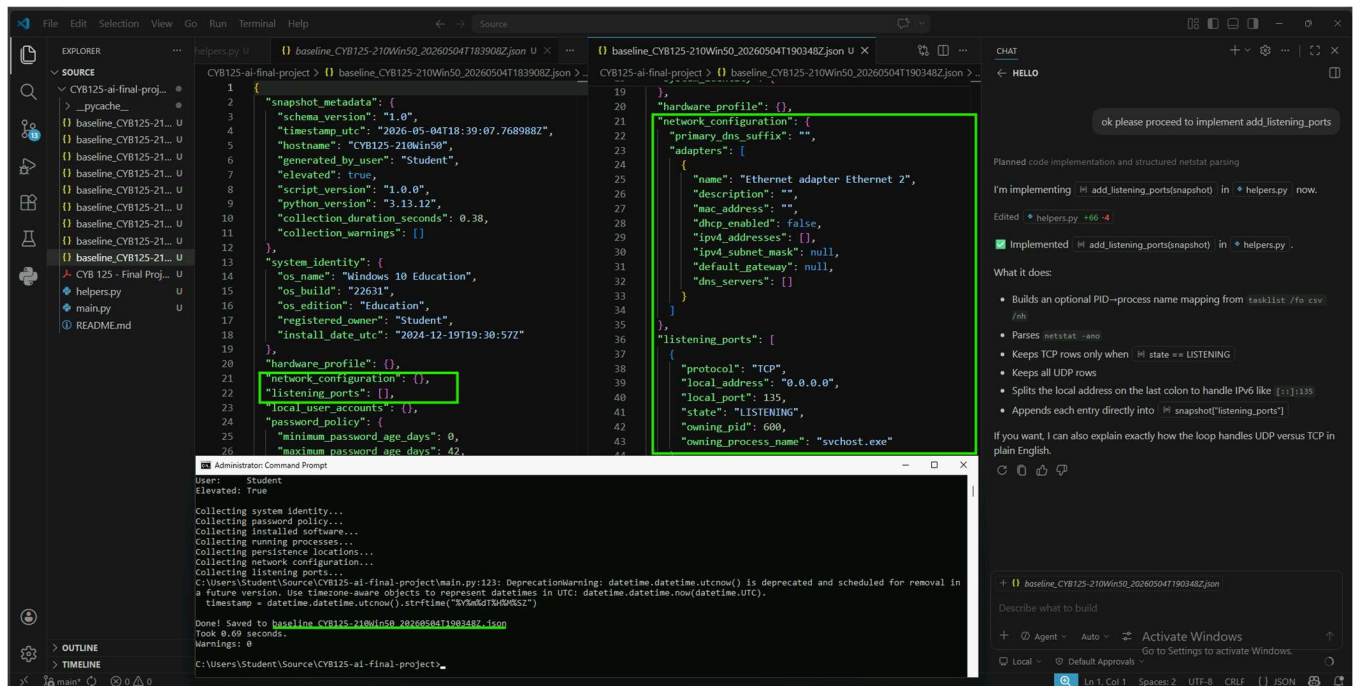
```
• git push origin main
```

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Part 2. Step 10 – Milestone #8: WMIC (Windows Management Instrumentation CLI):

[Requirement #29] – Copy and paste the following prompt into the Copilot Chat window. When Copilot has responded take a **screenshot** that includes your entire prompt and the AI response and paste it in a document that you will deliver with your project.

Prompt (copy and paste this): "I'm starting Milestone 8. Please uncomment the appropriate lines for milestone 8. I'm starting Milestone 8. I want to handle all seven in one pass. None of these introduce new Python concepts but several collectors call wmic ... /format:csv.

Finally, edit all seven empty stubs in place. Add an inline comment above each line of code explaining what it does and why. Where you make a deliberate choice that could have been done differently, call that out in the comment. Don't touch any function I've already implemented in earlier milestones. Use `run_command()` to invoke commands and `get_registry_value()` to read individual registry values;

After you produce the diff, give me a short summary at the end, one or two sentences per collector — describing what each one ended up doing. I'll use that summary as a checklist when I review the code section by section. Final note, do not use any Powershell calls."

If you are happy with Copilots changes, click on "Keep" otherwise click "Undo" and prompt Copilot again to make the corrections that you think are necessary.

[Requirement #30] – Answer the following questions:

1. The diff was long. How did you make sure each of the seven collectors was correct before you accepted the diff? Did you read every line, spot-check a few, run the code first and then read, or something else? There's no wrong answer, I want to know what your real process was. If you noticed any line you didn't fully understand, name it.
2. Save the changes Copilot made to your main.py, run it in your command prompt, and open the JSON file that was generated as output. All 16 sections of your JSON should now be populated, OR contain an entry in `collection_warnings` explaining why a section is empty. Open the JSON and confirm. List any sections that are empty without a corresponding warning — those are bugs. List any warnings that surprise you (warnings you didn't expect to see based on what you knew about your VM)?

[Requirement #31] – Once you confirm that the code generates the correct output:.

- In you command prompt, in your project folder, copy and paste the following commands:

- `git add *.py`

- This command will add all files with a .py extension to your submission

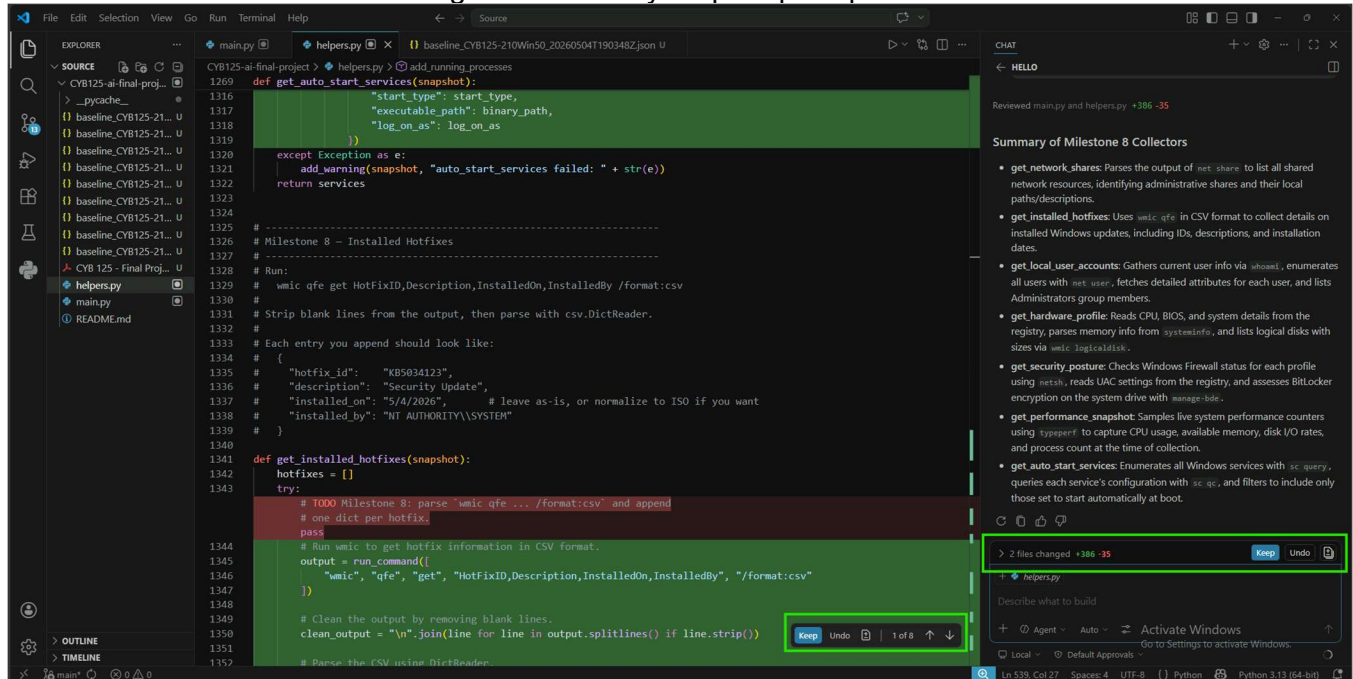
- `git commit -m "Milestone #8 WMIC"`

- This command will package your submission and relate all the files that have changed to a single entity called a "commit" in this case it's just one file.

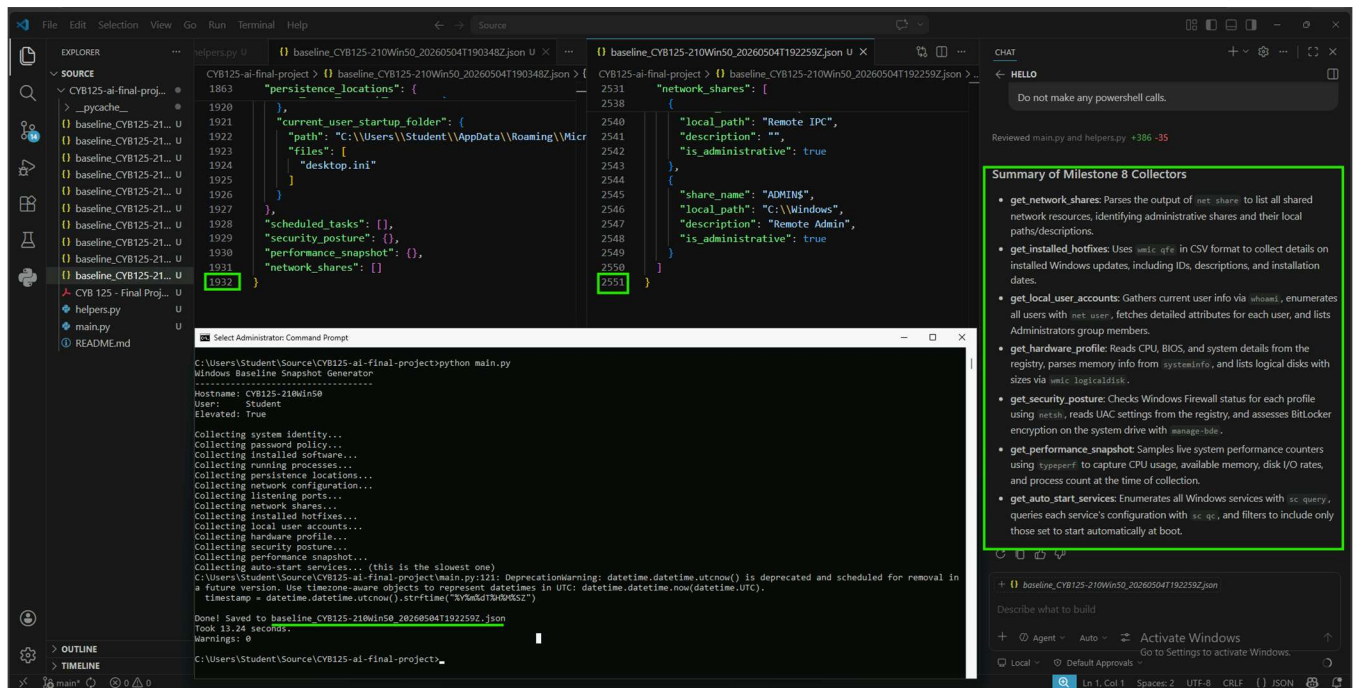
- `git push origin main`

- This command will publish your commit so that I can review it.

Your screen should look something like this after your prompt Copilot:



Your screen should look something like this after you run `python main.py` and compare the json output against the previous json file generated:



If running the script generated a runtime error you should copy and paste the error into the chat and ask Copilot to explain it to you, then suggest and implement a fix.

Part 3. Final Reflection (18% of your grade)

[Requirement 33] Please include all responses to Requirements in a separate document (docx or pdf) called <Lastname>-CYB125-Final-Responses.

[Requirement 34] Please include the URL to your github repository (the same link you provided in generated in Step 0.)

[Requirement 35] Choose any 3 applicable questions and respond with a minimum of 50 words to each:

1. How should professionals balance efficiency gained through AI with the responsibility to maintain deep technical competence? Based on your experience, where should the line be drawn between delegating tasks to AI and preserving human expertise?
2. In what ways did the act of specifying the program (in Part 1) shape your mental model of the problem? How did this internal model influence your ability to evaluate, challenge, or refine the code produced by the AI?
3. Which aspects of the project required distinctly human judgment that the AI could not replicate, and why? What risks arise in professional settings when developers outsource these types of decisions to generative models?
4. How did your ability (or inability) to interpret the AI's generated code influence your confidence in the final system? Based on your experience, why is it essential for developers to understand not just what AI generates, but how and why it works?
5. What did you learn about prompting strategy, prompt decomposition, or iterative refinement that you believe will generalize to more complex AI-assisted engineering tasks? Provide a concrete example.

[Requirement 36] Project Feedback Questionnaire

This questionnaire will be used only to improve the project for future students. There are no right or wrong answers.

1. How clearly were the project instructions and expectations explained?
(Unclear) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Perfectly Clear)
2. How much did this project help you understand functions, dictionaries, and lists in Python?
(No Understanding) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Strong Understanding)
3. How effectively did AI support your learning without replacing your own reasoning?
(Not Effective) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Highly Effective)

4. How appropriate was the difficulty level for your skill level?
(Too Easy / Too Hard) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Well-Suited to My Level)
5. How helpful was the milestone-by-milestone structure compared to a single all-at-once project?
(Not Helpful) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Extremely Helpful)
6. How confident are you in your ability to understand and explain your final script?
(Not Confident) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Very Confident)
7. How useful was the starter code for getting you over the "blank page" barrier?
(Not Useful) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Very Useful)
8. How useful was example_baseline.json as a reference for what you were building toward?
(Not Useful) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Very Useful)
9. How relevant did this project feel to a cybersecurity career path?
(Not Relevant) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Highly Relevant)
10. How valuable was this project for your development as a programmer?
(Not Valuable) 1 — 2 — 3 — 4 — 5 — 6 — 7 — 8 — 9 — 10 (Highly Valuable)
11. **Free response:** What's one thing about this project that worked really well for you, and one thing you'd change?

Grading

You will get an A (90%) on this project if you complete Part 1 + Milestones 1-5 and the final reflection. Milestone 6-8 are more interesting from a security perspective, complete these milestones to get a 100%. Each milestone is graded on three things:

1. the matching JSON section is populated when I run your script,
2. you committed and pushed at the end of that milestone with the suggested commit message (or a clear equivalent), and
3. the milestone reflection is present and addresses the question. Partial credit is available for partial implementations.
4. You will need to commit and push to GitHub after every milestone. Your git history is part of your grade and I want to see the snapshot grow over time, not appear all at once.

Component	Points
Part 1 — Project Plan + data dictionary skeleton	50 pts
Part 2 — Milestones 1-5 (core required milestones)	25 x 5 = 100 pts
Part 2 — Milestones 6-8 (stretch milestones)	10 x 3 = 30 pts
Part 3 — Final Reflection	50 pts
TOTAL	300 pts

Due: Please submit a **FINAL DRAFT** by **Monday, 5/11 @ 11:59 pm**. Monday is the last official day of the semester so no late work will be able to be accepted after that date and time.

Please contact me ASAP if you have any questions or would like any assistance!